

Chapter 8-1

Target Code Generation

PART I: Target Code Generation

Target Machine

- Memory (Heap/Stack/...). Each byte has an address.
 - n Registers: $R0, R1, \dots, R_{n-1}$. Each four bytes.
 - Load/Store/Calculation/Jump/... (Like x86 assembly)
-
- | | | |
|------------------|--|-------------------------------------|
| • Load/Store | • Calculation | • Jump |
| • LD $R0, addr$ | • OP dst, src_1, src_2 | • BR $L/addr$ |
| • LD $R0, R1$ | • <i>e.g.</i> , SUB $R0, R1, R2$ | • Bcond $R, L/addr$ |
| • LD $R0, \#500$ | | • <i>e.g.</i> , B LTZ R, L |
| • ST $addr, R0$ | (Each LD/ST loads/stores a 4-byte integer) | |

Target Machine

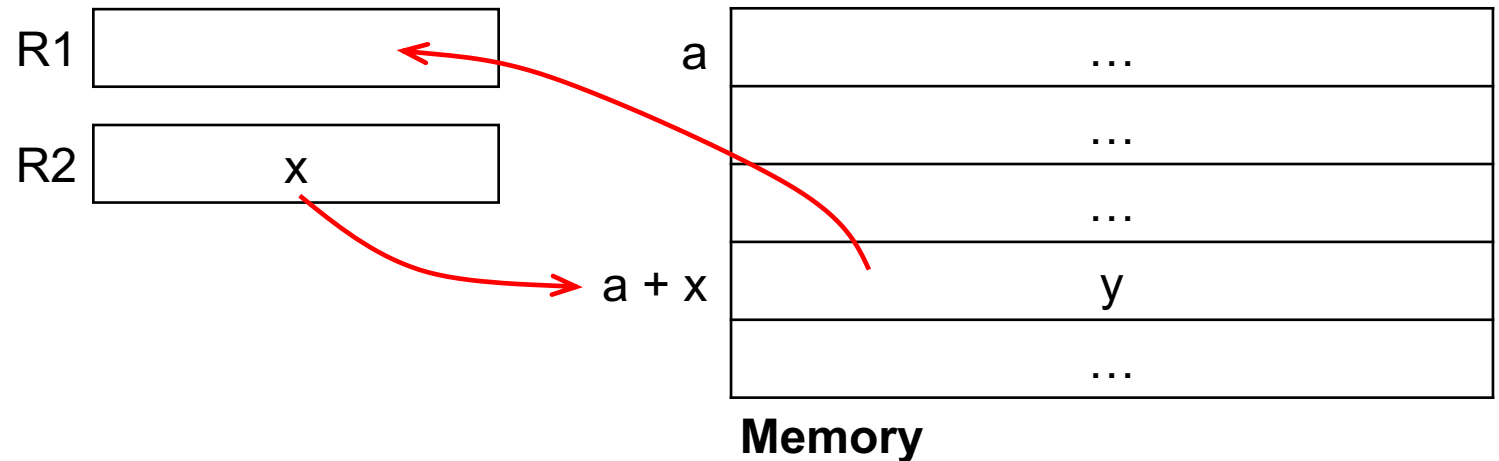
- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

$$x = y - z$$



Target Machine

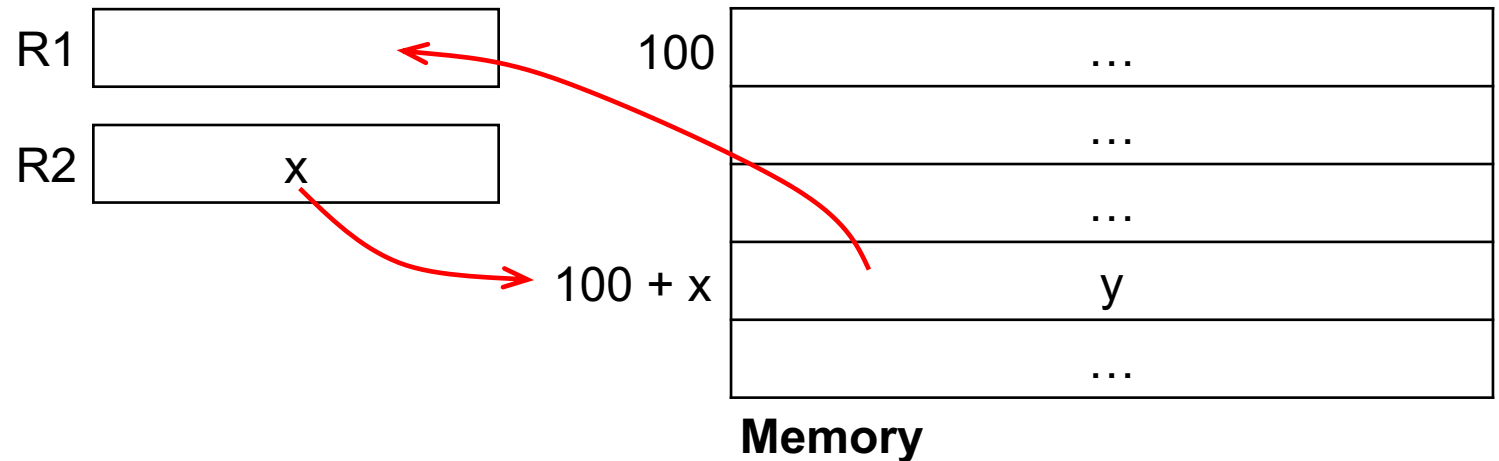
- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

$$x = y - z$$



Target Machine

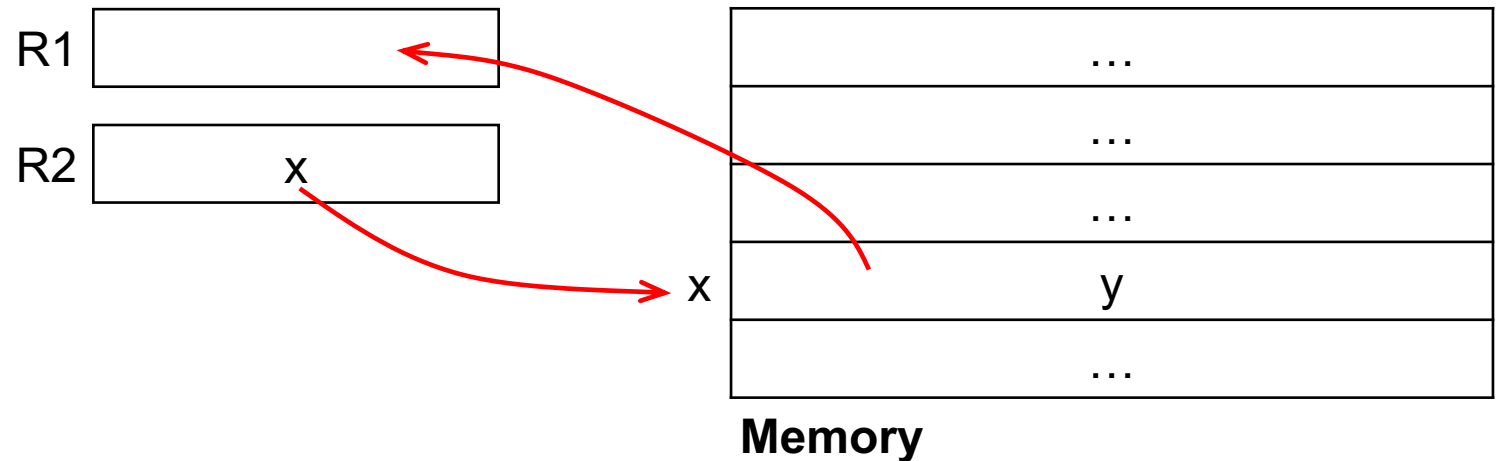
- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

$$x = y - z$$



Target Machine

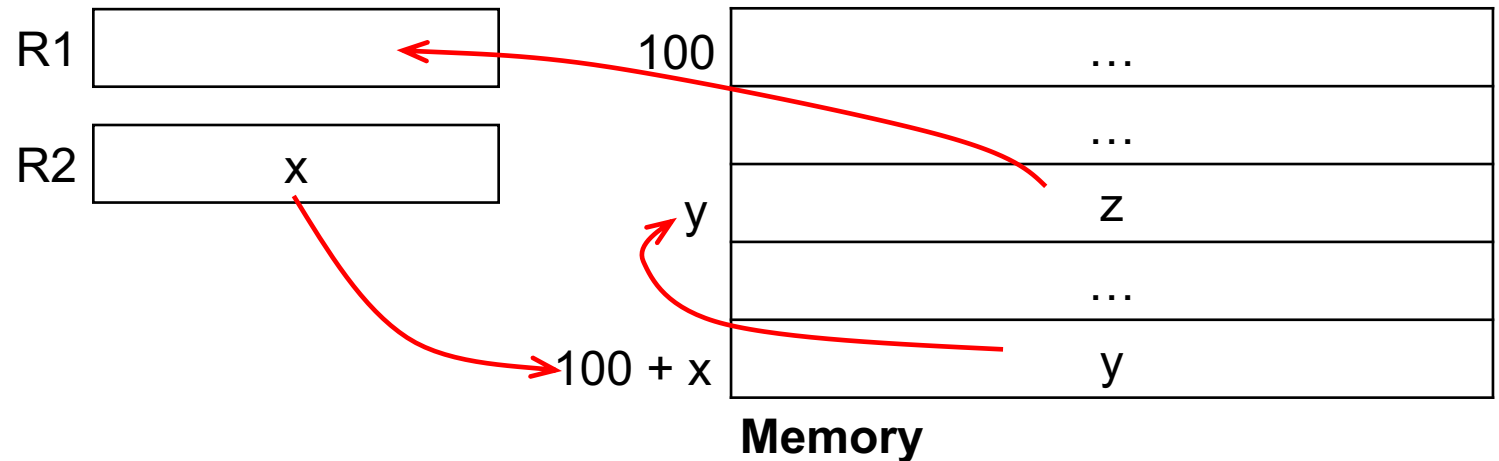
- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

$$x = y - z$$



Target Machine

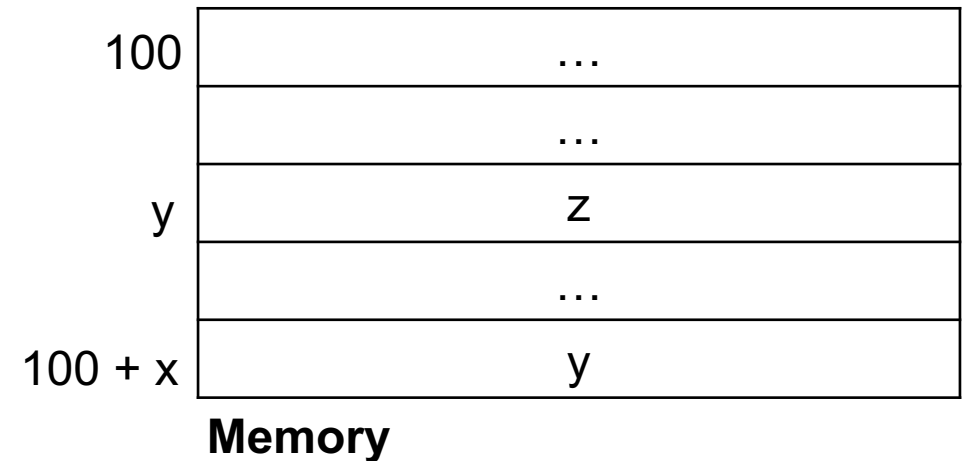
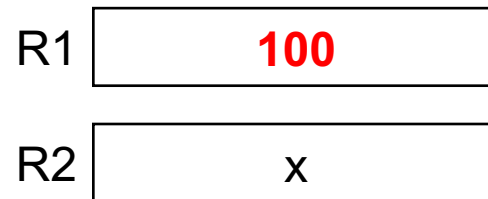
- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2       // R1 = R1 - R2
ST  x, R1           // x = R1
```

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

$$x = y - z$$



Target Machine

- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

x = y - z

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

Addressing Modes: How we compute the addr

- LD R, **addr**
- ST **addr**, R
- BR **addr**

Target Machine

- Load/Store/Calculation/Jump/...

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

- Addressing Modes

- LD R1, a(R2)
- LD R1, 100(R2)
- LD R1, *R2
- LD R1, *100(R2)
- LD R1, #100

x = y - z

```
LD  R1, x           // R1 = x
LD  R2, y           // R2 = y
SUB R1, R1, R2      // R1 = R1 - R2
BLTZ R1, M          // if R1 < 0 jump to M
```

if x < y goto M

Address Representation

**Q: How does a machine understand
x, y, z, and M?**

**A: Each variable or label corresponds to
a memory address**

```
LD  R1, y      // R1 = y
LD  R2, z      // R2 = z
SUB R1, R1, R2  // R1 = R1 - R2
ST  x, R1      // x = R1
```

x = y - z

```
LD  R1, x      // R1 = x
LD  R2, y      // R2 = y
SUB R1, R1, R2  // R1 = R1 - R2
BLTZ R1, M     // if R1 < 0 jump to M
```

if x < y goto M

Memory Structure

0100	Code/Text
...	
0400	Global/Static
...	
0800	Heap
...	
...	
...	Idle
...	
5000	Stack
...	
...	

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2       // R1 = R1 - R2
ST  x, R1           // x = R1
```

x = y - z

```
LD  R1, x           // R1 = x
LD  R2, y           // R2 = y
SUB R1, R1, R2       // R1 = R1 - R2
BLTZ R1, M           // if R1 < 0 jump to M
```

if x < y goto M

Text Memory

0100	LD R1 x
0104	LD R2 y
0108	SUB R1 R1 R2
0112	BLTZ R1 #0124
0116	...
0120	...
0124	...
...	...
	Static
	Heap
	Idle
	Stack

M is translated to a memory address

```
LD    R1, x           // R1 = x
LD    R2, y           // R2 = y
SUB   R1, R1, R2      // R1 = R1 - R2
BLTZ  R1, M           // if R1 < 0 jump to M
```

if x < y goto M

Global Memory

0100	LD R1, *0400	Text
0104	LD R2, *0404	
0108	SUB R1, R1, R2	
0112	...	
...	...	
0400	2	Global
0404	3	
0408	...	
...	...	
...		Heap
...		Idle
...		Stack

```
LD  R1, y      // R1 = y
LD  R2, z      // R2 = z
SUB R1, R1, R2 // R1 = R1 - R2
ST  x, R1      // x = R1
```

```
int y = 2;
int z = 3;
```

```
void foo () {
    int x;
    x = y - z;
}
```

x = y - z

Take Away Message

The memory locations of **the code** and **the globals** are determined by the compiler!

The memory locations of other variables, *e.g.*, **the locals**, are controlled by the code generated by the compiler and stored in heap or stack!

Stack Memory

0100	Code/Text
...	
0400	Global/Static
...	
0800	Heap
...	
...	
...	Idle
...	
5000	Stack
...	
...	

Recap: Global Memory

0100	LD R1, *0400	Text
0104	LD R2, *0404	
0108	SUB R1, R1, R2	
0112	ST x, R1	
...	...	
0400	2	Global
0404	3	
0408	...	
...	...	
...		Heap
...		Idle
...		Stack

```
LD R1, y      // R1 = y
LD R2, z      // R2 = z
SUB R1, R1, R2 // R1 = R1 - R2
ST x, R1      // x = R1
```

x = y - z

```
int y = 2;
int z = 3;
```

```
void foo () {
    int x;
    x = y - z;
}
```

Stack Memory

0100	LD R1 *0400	Text
0104	LD R2 *0404	
0108	SUB R1 R1 R2	
0112	ST x , R1	
...	...	
0400	2	Global
0404	3	
0408	...	
...	...	
...		Heap
...		Idle
...		Stack

**The location of x depends
on the call stack!**

```
LD  R1, y      // R1 = y
LD  R2, z      // R2 = z
SUB R1, R1, R2 // R1 = R1 - R2
ST  x, R1     // x = R1
```

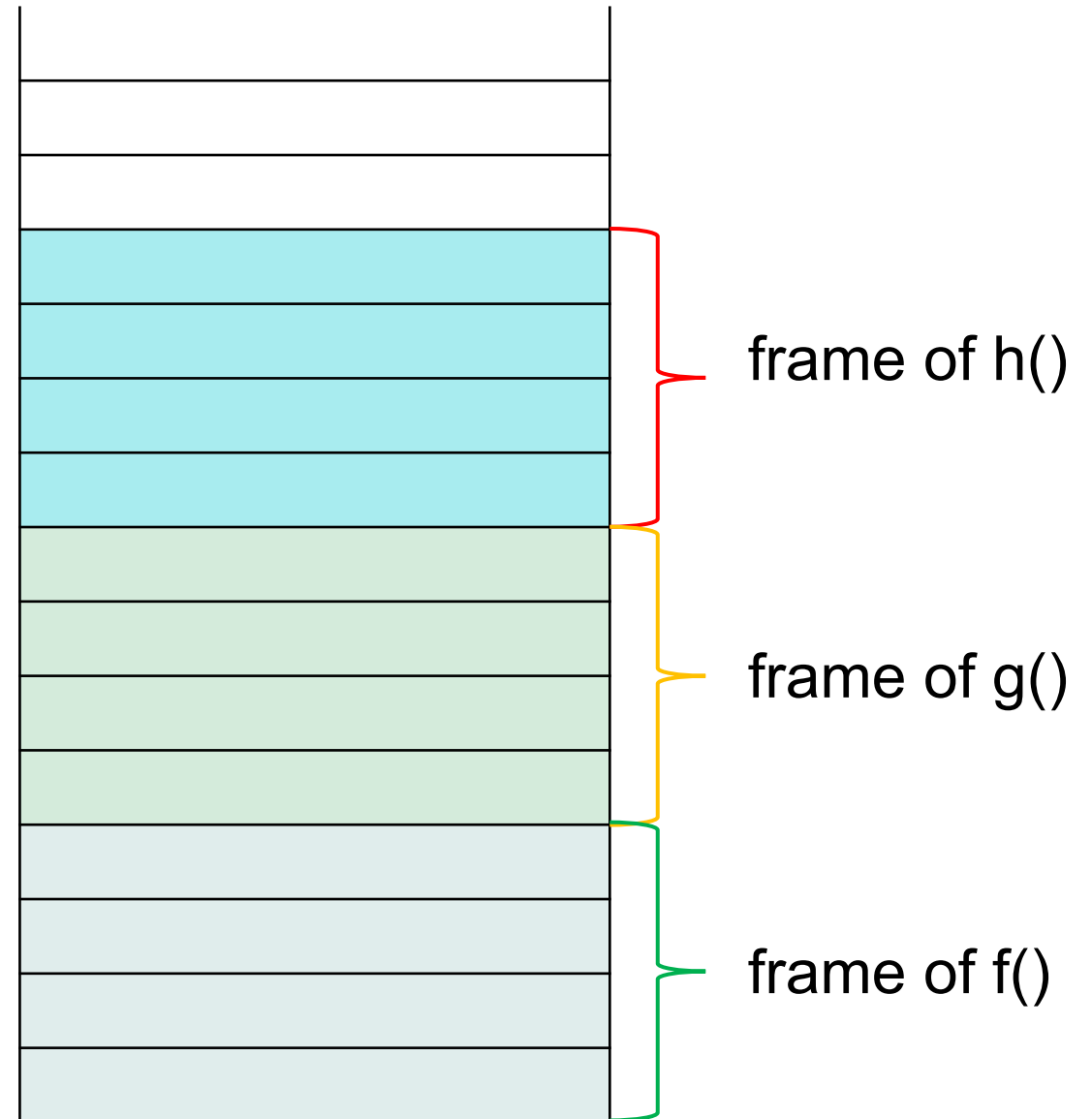
x = y - z

```
void foo () {
    int x;
    x = y - z;
}
```

Stack Memory

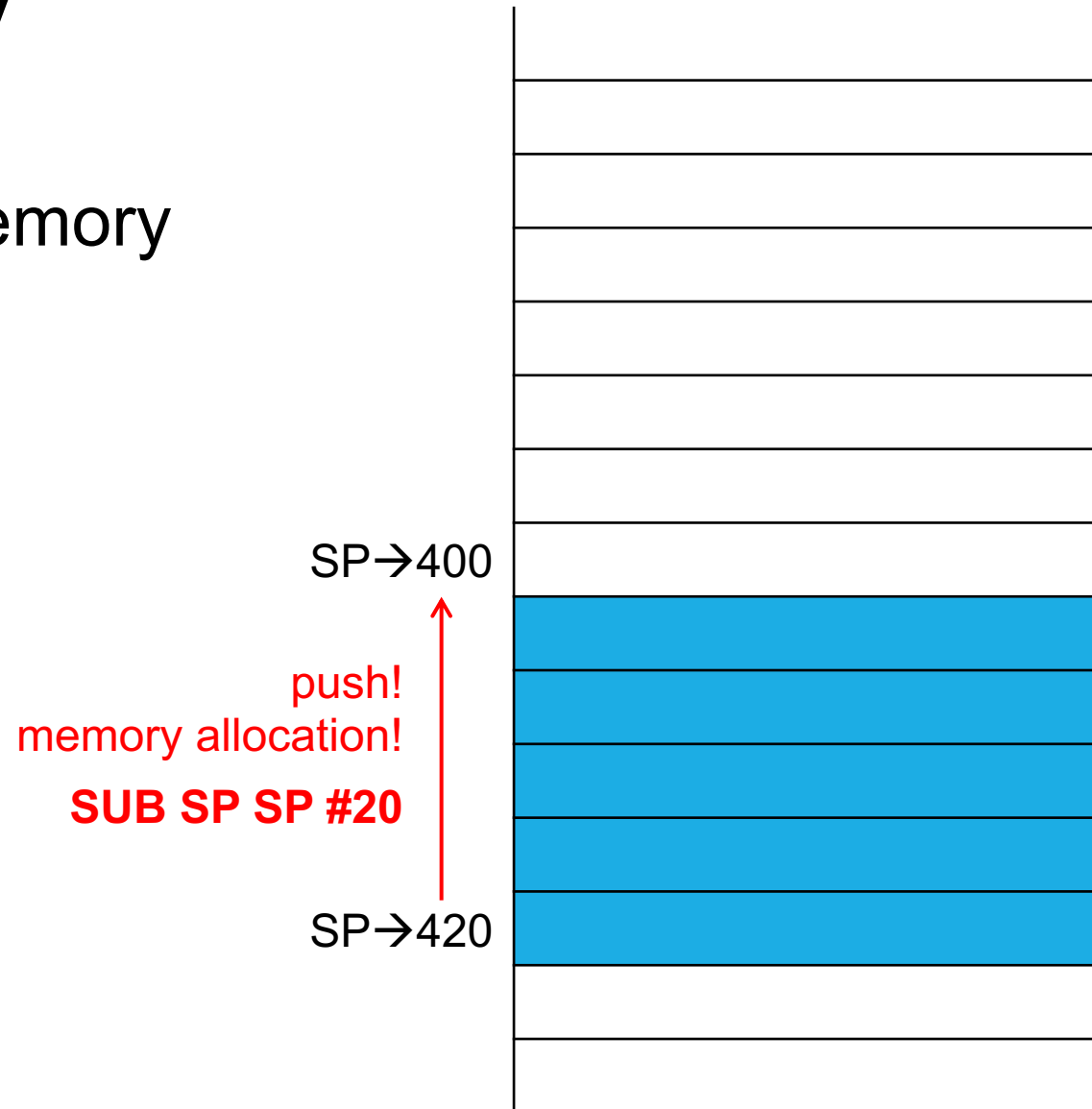
- Call Stack
- What is in a Frame?
 - Local variables
 - Return address
 - Return value, Parameters
 - Data to recover
 - ...
- **A special register: SP**
- **A convention** of using stack

Stack



Stack Memory

- Push or Allocate Stack Memory



Stack Memory

- Before a call
 - If the procedure uses some registers, push them into the stack
 - Push (record) the **return address** & arguments to the stack
 - Jump to the code of callee
- During a call
 - Allocate memory spaces for locals; and actions based on the locals
- After a call
 - Pop the frame;
 - Return to the caller according to the **return address** in the stack
 - Recover data; Get the return value if the function has a return value

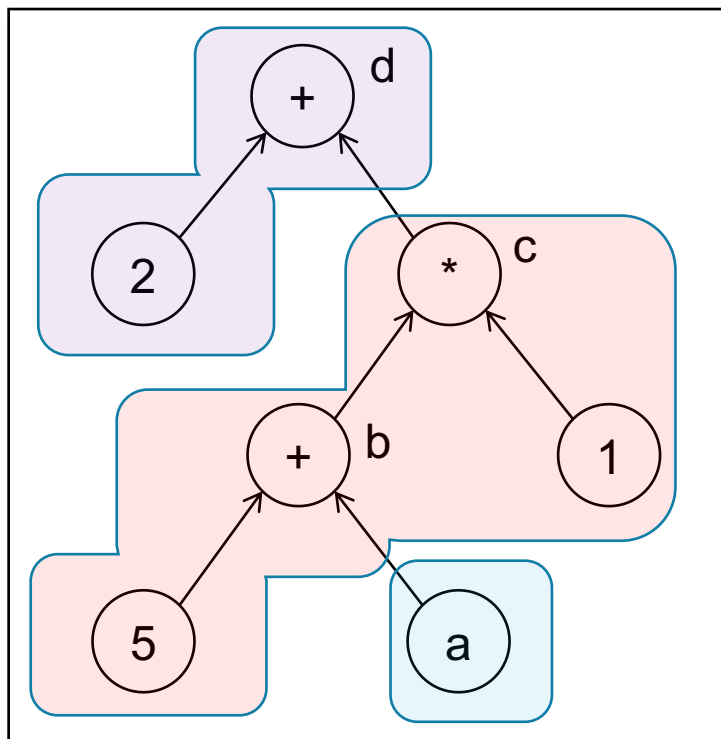
Heap Memory

- Dynamically allocate memory in heap
 - C: `void *malloc(size_t)`
 - Java/C++: the **new** operator
- Focus on the efficiency and minimizing segmentation
- Manual management (C/C++) or garbage collection (Java)
- ***Refer to Chapter 7, the Dragon book.***

PART II-1: In-Block Optimization

Tree-Based Instruction Selection

$b = a + 5$
 $c = b * 1$
 $d = 2 + c$



LD	R0,	a
LD	R1,	5
ADD	R2,	R0, R1
ST	b,	R2
LD	R3,	1
MUL	R4,	R2, R3
ST	c,	R4
LD	R5,	2
ADD	R6,	R4, R5
ST	d,	R6

LD	R0,	a
ADD	R1,	R0, 5
ST	b,	R1
ST	c,	R1
ADD	R2,	R1, 2
ST	d,	R2

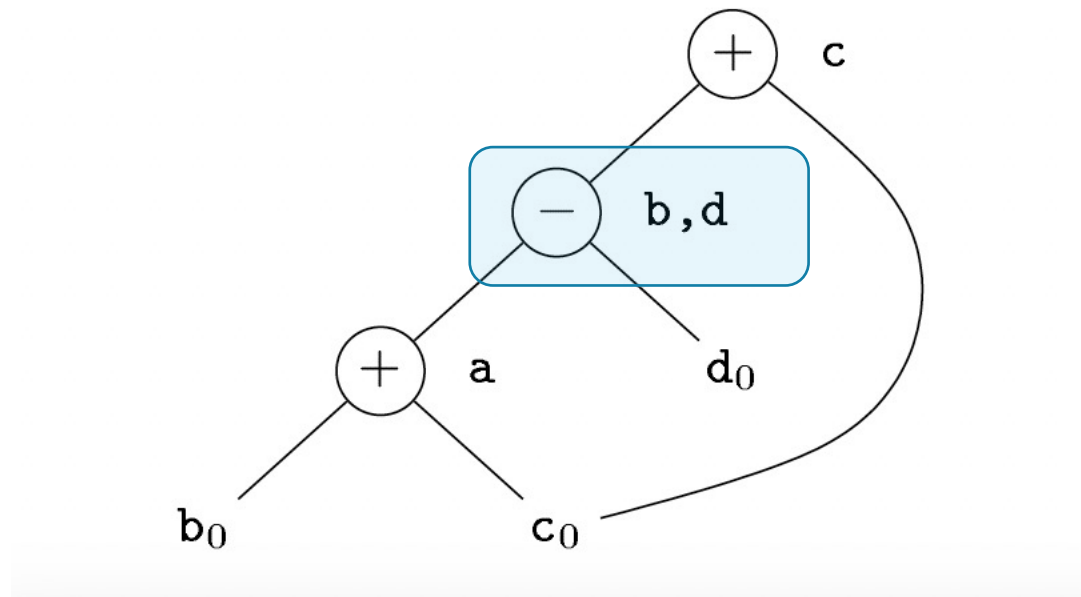
Tree-Based Instruction Selection

- Define a series of rules for selecting instructions
 - Pattern $\rightarrow$ Instructions
- **The effectiveness depends on the rules predefined.**

Local Common Sub-Expression

```
a = b + c
b = a - d
c = b + c
d = a - d
```

It can be replaced by $d = b$.



Local Common Sub-Expression

- $a = x - y$
- if $x \geq y$ goto L
- =====>
- $a = x - y$
- if $x - y \geq 0$ goto L
- =====>
- $a = x - y$
- if $a \geq 0$ goto L

Algebraic Identities

$$x + 0 = 0 + x = x$$

$$x \times 1 = 1 \times x = x$$

$$x - 0 = x$$

$$x/1 = x$$

EXPENSIVE

CHEAPER

$$x^2$$

=

$$x \times x$$

$$2 \times x$$

=

$$x + x$$

$$x/2$$

=

$$x \times 0.5$$

Machine Idioms

```
LD  R0, a      // R0 = a
ADD R0, R0, #1  // R0 = R0 + 1
ST  a, R0      // a = R0
```

How about “INC a” ??

Possible target code for $a = a + 1$

Redundant Loads/Stores

```
LD R0, a
ST a, R0
```

PART II-2: Peephole Optimization

Peephole Optimization

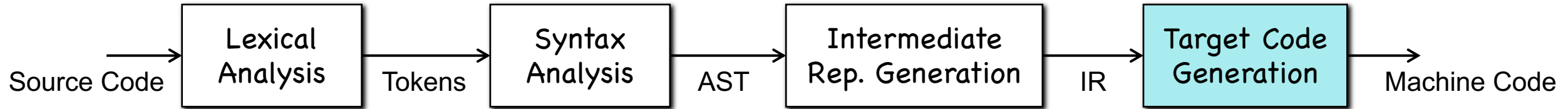
```
1)  i = 1
2)  j = 1
3)  t1 = 10 * i
4)  t2 = t1 + j
5)  t3 = 8 * t2
6)  t4 = t3 - 88
7)  a[t4] = 0.0
8)  j = j + 1
9)  if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

- Optimize the code in a small window
- Work on IR or the target code
- **Cross-block optimization**

Chapter 8-2

Translation out of SSA

Target Code Generation

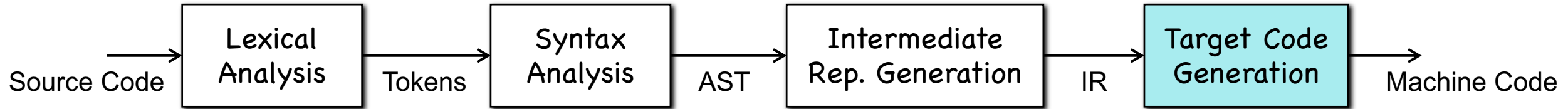


What challenges do we have to address, so as to translate SSA IR (e.g., **LLVM IR**) to machine code?

The ϕ functions!

How can we translate (or eliminate) the ϕ functions?

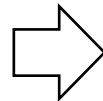
Target Code Generation



Solution 1: Since ϕ merge values of the same variable, let's replace variables merged by ϕ with the same variable.

```

x0 = 1;
do {
    x1 =  $\phi$ (x0, x2);
    x2 = x1 + 1;
} while(...);
print(x1);
  
```

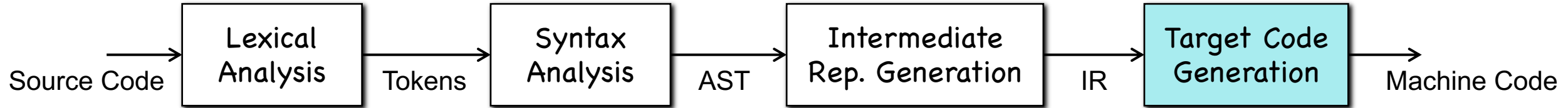


```

x = 1;
do {
    x1 =  $\phi$ (x0, x2);
    x = x + 1;
} while(...);
print(x);
  
```



Target Code Generation

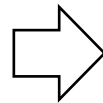


Solution 2: Replace ϕ function with copies in predecessor blocks.



```

x0 = 1;
do {
    x1 =  $\phi(x0, x2)$ ;
    x2 = x1 + 1;
} while (...);
print(x1);
  
```

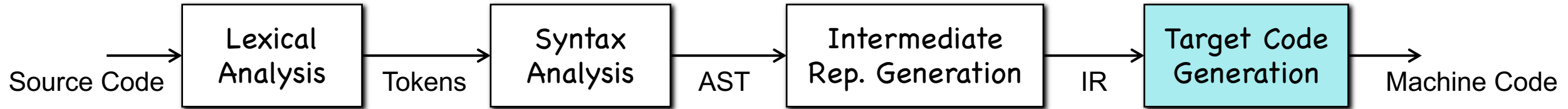


```

x0 = 1; x1 = x0;
do {
    x1 =  $\phi(x0, x2)$ ;
    x2 = x1 + 1;
    x1 = x2;
} while (...);
print(x1);
  
```

The Lost Copy Problem!

Target Code Generation

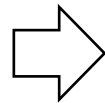


Solution 2: Replace ϕ function with copies in predecessor blocks.



```

x0 = 1; y0 = 2;
do {
    x1 =  $\phi$ (x0, y1);
    y1 =  $\phi$ (y0, x1);
} while (...);
print(x1, y1);
  
```

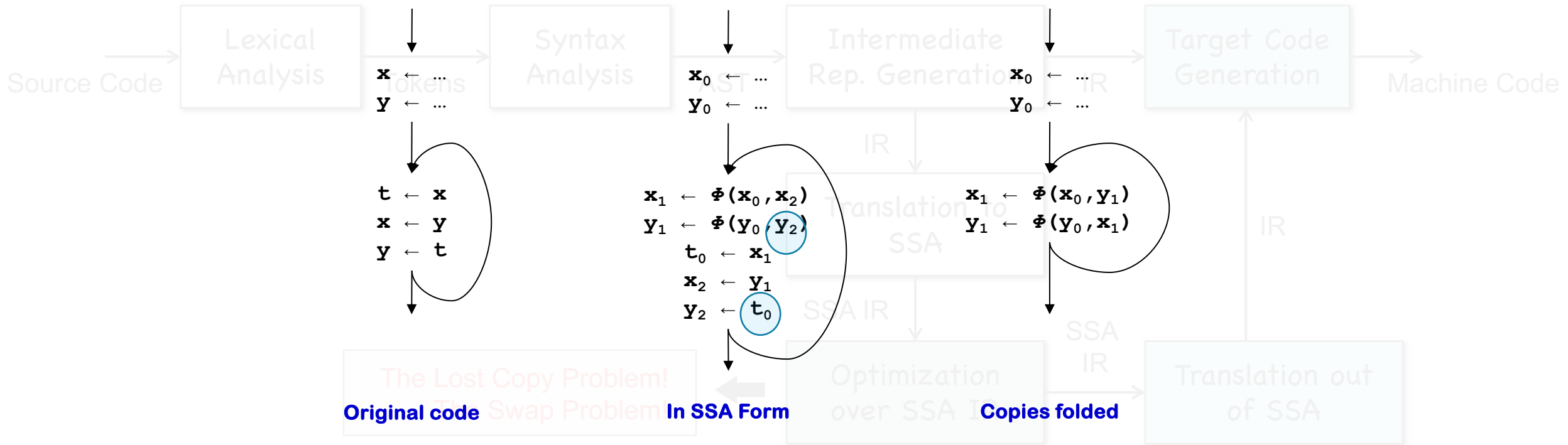


```

x0 = 1; y0 = 2;
x1 = x0; y1 = y0;
do {
    x1 = y1;
    y1 = x1;
} while (...);
print(x1, y1);
  
```

The Swap Problem!

Where're the Problems from?



The Lost Copy Problem

- The copy $X1 = X0$ is lost!
- To address the issue, we need to create a temp variable to hold this value.

```
x0 = 1;
do {
    x1 =  $\phi$ (x0, x2);
    x2 = x1 + 1;
} while (...);
print(x1);
```

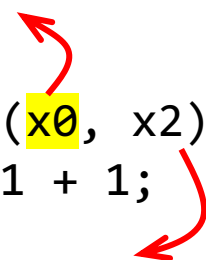
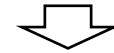


```
x0 = 1; x1 = x0;
do {
    x1 =  $\phi$ (x0, x2);
    x2 = x1 + 1;
    x1 = x2;
} while (...);
print(x1);
```

The Lost Copy Problem

- For each $a = \phi(a_1, a_2, \dots)$
 - Insert $a_i' = a_i$ at the end of the block of a_i
 - Replace the ϕ with $a' = \phi(a_1', a_2', \dots)$
 - Insert a copy of $a = a'$ after ϕ
- Replace all a' and a_i' with a new name
- Remove ϕ by inserting copies in predecessors
- Coalesce redundant copies

```
x0 = 1;
do {
    x1 =  $\phi(x0, x2)$ ;
    x2 = x1 + 1;
} while (...);
print(x1);
```

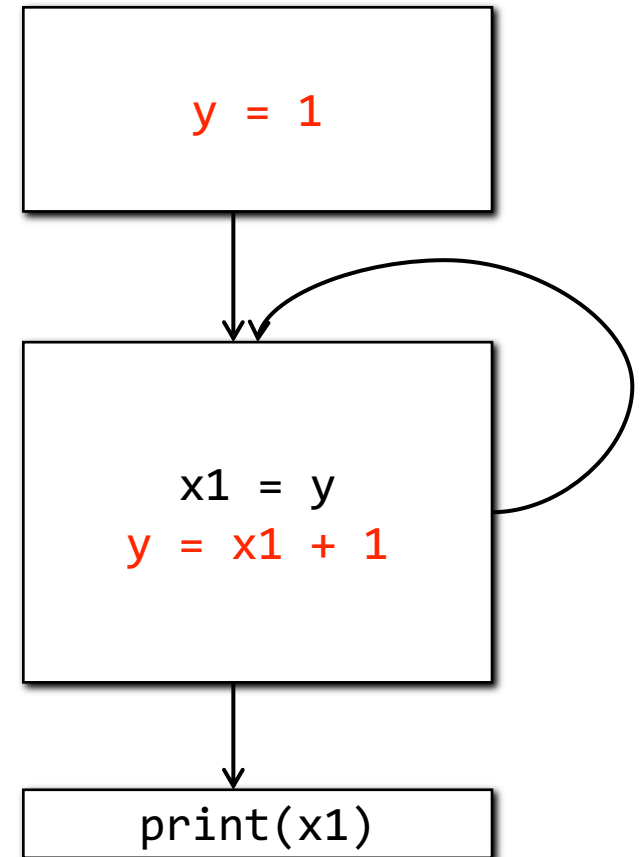



```
x0 = 1; x1 = x0;
do {
    x1 =  $\phi(x0, x2)$ ;
    x2 = x1 + 1;
    x1 = x2;
} while (...);
print(x1);
```

The Lost Copy Problem

- For each $a = \phi(a_1, a_2, \dots)$
 - Insert $a_i' = a_i$ at the end of the predecessor block
 - Replace the ϕ with $a' = \phi(a_1', a_2', \dots)$
 - Insert a copy of $a = a'$ after ϕ
- Replace all a' and a_i' with a new name
- Remove ϕ by inserting copies in predecessors
- Coalesce redundant copies

```
x0 = 1;
do {
    x1 =  $\phi(x0, x2)$ ;
    x2 = x1 + 1;
} while (...);
print(x1);
```

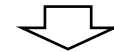


The Swap Problem

- Caused by multiple ϕ in the same block
- The parallel semantics of ϕ statements

```
x0 = 1; y0 = 2;

do {
    x1 =  $\phi$ (x0, y1);
    y1 =  $\phi$ (y0, x1);
} while (...);
print(x1, y1);
```



```
x0 = 1; y0 = 2;
x1 = x0; y1 = y0;
do {
    x1 = y1;
    y1 = x1;
} while (...);
print(x1, y1);
```

The Swap Problem

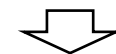
- For each $a = \phi(a1, a2, \dots)$
 - Insert $ai' = ai$ at the end of the block of ai
 - Replace the ϕ with $a' = \phi(a1', a2', \dots)$
 - Insert a copy of $a = a'$ after ϕ
- Replace all a' and ai' with a new name
- Remove ϕ by inserting copies in predecessors
- Sequentialize the parallel copies
- Coalesce redundant copies

```

x0 = 1; y0 = 2;

do {
    x1 =  $\phi$ (x0, y1);
    y1 =  $\phi$ (y0, x1);
} while (...);
print(x1, y1);

```



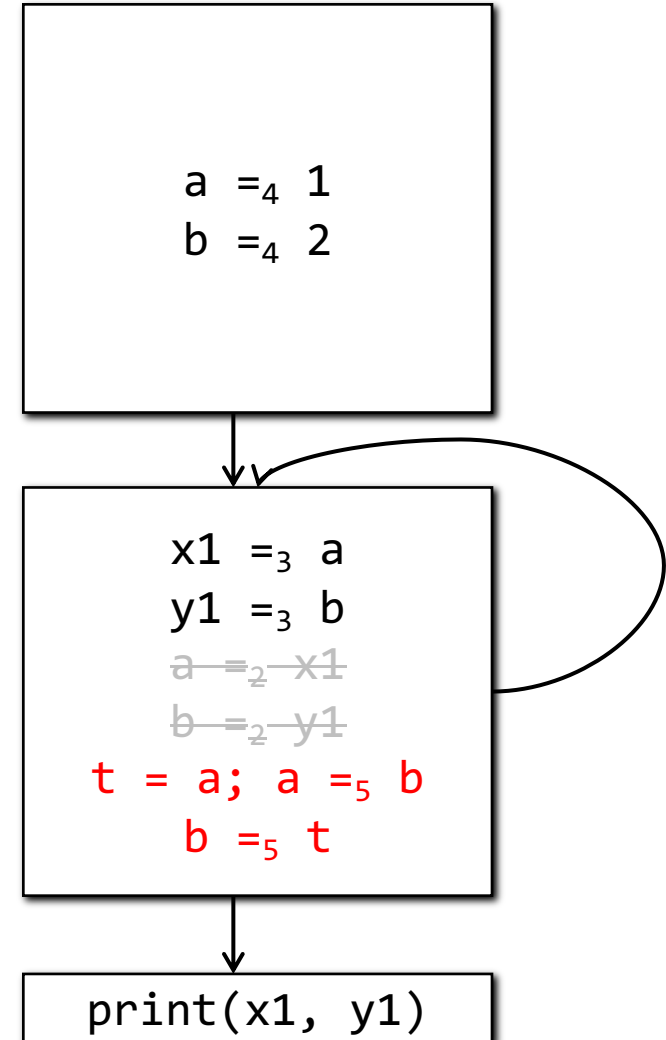
```

x0 = 1; y0 = 2;
x1 = x0; y1 = y0;
do {
    x1 = y1;
    y1 = x1;
} while (...);
print(x1, y1);

```

The Swap Problem

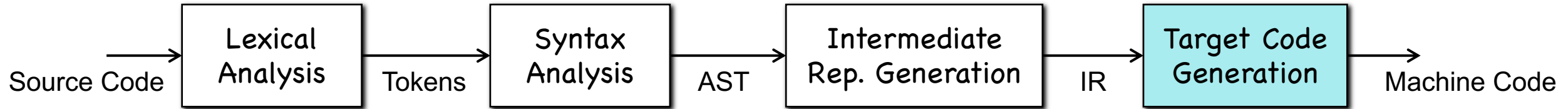
- For each $a = \phi(a1, a2, \dots)$
 - Insert $ai' = ai$ at the end of the block of ai
 - Replace the ϕ with $a' = \phi(a1', a2', \dots)$
 - Insert a copy of $a = a'$ after ϕ
- Replace all a' and ai' with a new name
- Remove ϕ by inserting copies in predecessors
- Sequentialize the parallel copies
- Coalesce redundant copies



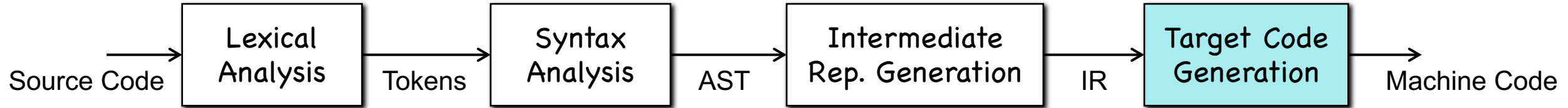
Chapter 8-3

Register Allocation

Register Allocation



Register Allocation



- Speed: Registers > Memory
- Physical machines have limited number of registers
- **Register allocation:** ∞ virtual registers $\rightarrow$ k physical registers

PART I: Local Register Allocation

Register Allocation

- Speed: Registers > Memory
- Physical machines have limited number of registers
- **Register allocation:** ∞ virtual registers $\rightarrow$ k physical registers
- **Requirement:**
 - Produce correct code using k or fewer registers
 - Minimize loads, stores, and space to hold spilled values
 - Efficient register allocation ($O(n)$ or $O(n \log n)$)

Register Allocation

- Two values CANNOT be mapped to the same register wherever they are both live (before their last use)
- **Spilling:**
 - saves a value from a register to memory
 - the register is then free for other values
 - we can load the spilled value from memory to register when necessary

MAXLIVE

1. LD	R1	#1028		// R1 <- 1028
2. LD	R2	*R1		// R2 <- contents(R1), assume y
3. MUL	R3	R1	R2	// R3 <- 1028 * y
4. LD	R4	x		// R4 <- x
5. SUB	R5	R4	R2	// R5 <- x-y
6. LD	R6	z		// R6 <- z
7. MUL	R7	R5	R6	// R7 <- z * (x-y)
8. SUB	R8	R7	R3	// R8 <- z * (x-y) - 1028 * y
9. ST	*R1	R8		// contents(R1) <- z * (x-y) - 1028 * y

MAXLIVE

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R5	R4 R2	// R1 R3 R5
6. LD	R6	z	// R1 R3 R5 R6
7. MUL	R7	R5 R6	// R1 R3 R7
8. SUB	R8	R7 R3	// R1 R8
9. ST	*R1	R8	//

MAXLIVE = 4

Enough to have 4 registers

Local Register Allocation

1. LD	R1	#1028	// R1
2. LD	R2	*R1	// R1 R2
3. MUL	R3	R1 R2	// R1 R2 R3
4. LD	R4	x	// R1 R2 R3 R4
5. SUB	R2	R4 R2	// R1 R3 R2
6. LD	R4	z	// R1 R3 R2 R4
7. MUL	R2	R2 R4	// R1 R3 R2
8. SUB	R2	R2 R3	// R1 R2
9. ST	*R1	R2	//

MAXLIVE = 4

if $k \geq 4$, e.g., $k = 4$

Local Register Allocation

1. LD	R1	#1028	// R1	
2. LD	R2	*R1	// R1 R2	
3. MUL	R3	R1 R2	// R1 R2 R3	Spill R3 by <u>ST v R3</u>
4. LD	R3	x	// R1 R2 R3	
5. SUB	R5	R3 R2	// R1 R5	MAXLIVE = 4
6. LD	R6	z	// R1 R5 R6	if k < 4, e.g., k = 3
7. MUL	R7	R5 R6	// R1 R7	
8. SUB	R8	R7 R3	// R1 R8	Reload R3 by <u>LD R3 v</u>
9. ST	*R1	R8	//	

Spill Value, Next Use, Farthest

1. LD	R1	#1028		// R1	
2. LD	R2	*R1		// R1 R2	
3. MUL	R3	R1	R2	// R1 R2 R3	Spill R1 by <u>ST v R1</u>
4. LD	R4	x		// R1 R2 R3 R4	
5. SUB	R5	R4	R2	// R1 R3 R5	
6. LD	R6	z		// R1 R3 R5 R6	
7. MUL	R7	R5	R6	// R1 R3 R7	
8. SUB	R8	R7	R3	// R1 R8	
9. ST	*R1	R8		//	

MAXLIVE = 4
if $k < 4$, e.g., $k = 3$

PART II: Global Register Allocation

Global Register Allocation

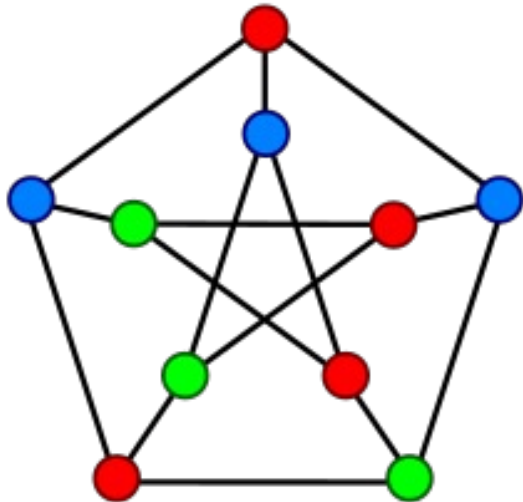
- Local register allocation does not capture reuse of values across multiple basic blocks
- Global allocation often uses the **graph-coloring** paradigm
 - Build a **conflict/interference graph**
 - Find a **k-coloring** for the graph, or change the code to a nearby problem that it can color
 - **NP-complete** under nearly all assumptions, so heuristics are needed

K-Coloring

- **Vertex Coloring:** assign a color to each vertex such that no edge connects vertices with the same color.
- **K-Coloring:** a coloring using at most k colors

K-Coloring

- **Vertex Coloring:** assign a color to each vertex such that no edge connects vertices with the same color.
- **K-Coloring:** a coloring using at most k colors



K-Coloring for Register Allocation

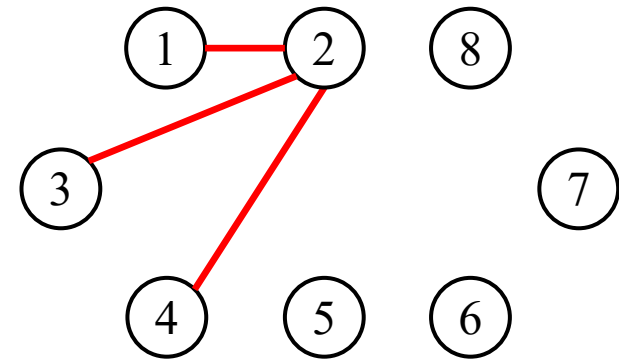
- Map graph vertices onto virtual registers
- Map colors onto physical registers
- From live ranges construct a **conflict graph**
- Color the graph so that no two neighbors have the same color
- If graph needs more than k colors – Spilling

Conflict Graph

- **Vertex:** virtual registers;
- **Edges:** virtual registers that have overlapping live range

If we can use k , e.g., 4, colors, to color the graph, the 8 virtual registers can be replaced by 4 physical registers.

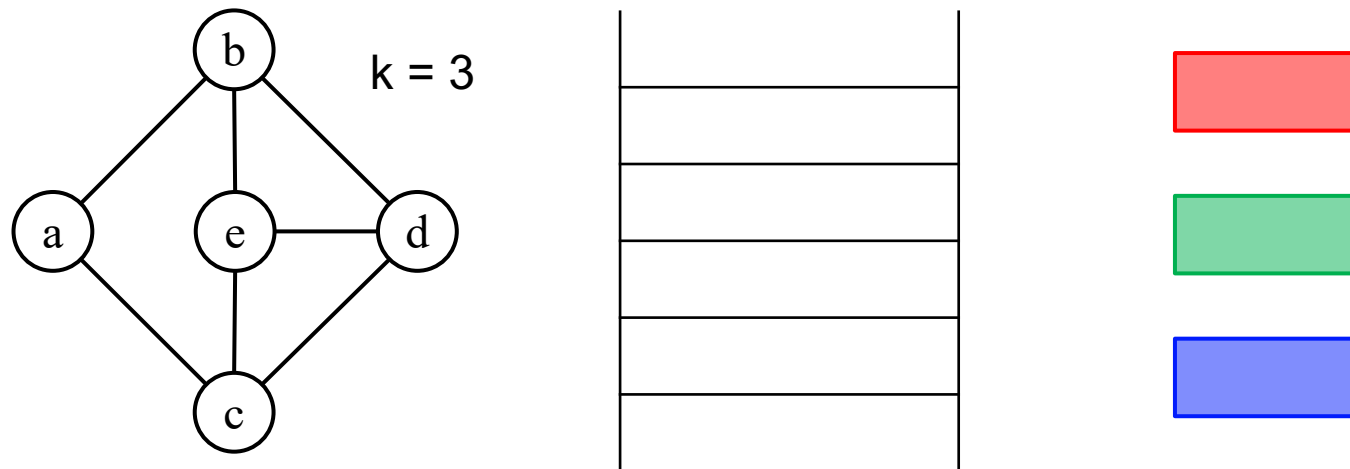
If we have to use 5 colors to color the graph, but only $k = 4$ physical registers are available, spilling is necessary.



Conflict Graph

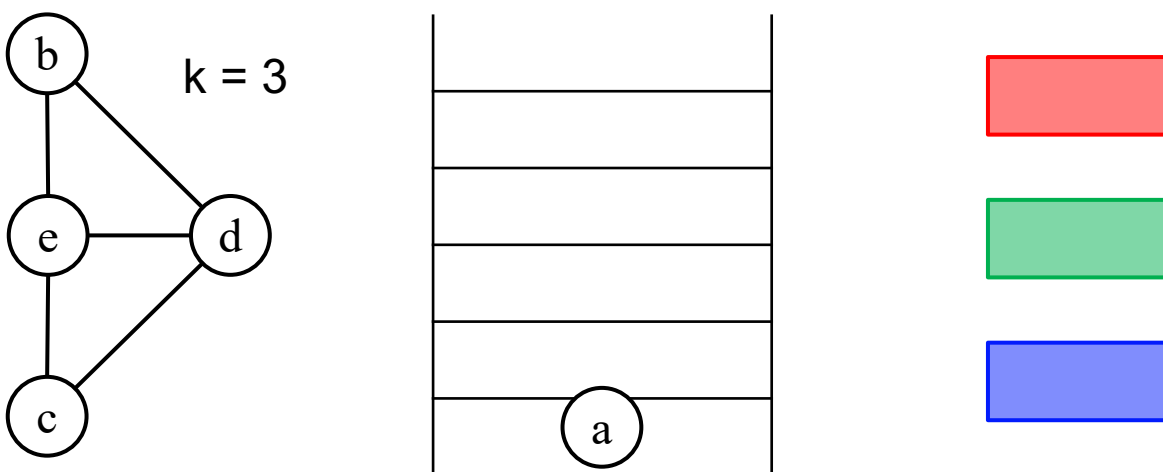
Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



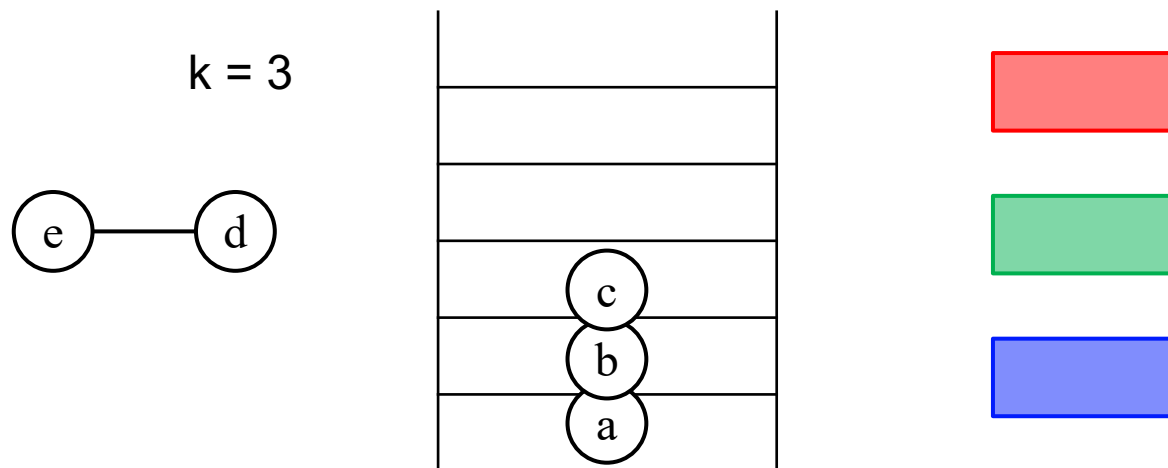
Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



Chaitin's Algorithm

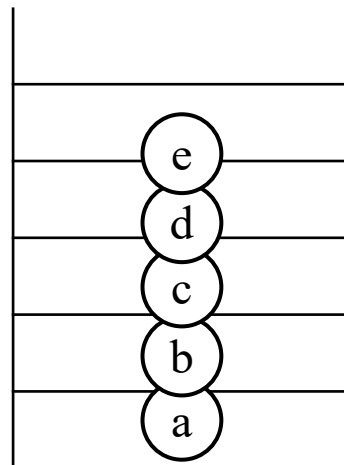
- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



Chaitin's Algorithm

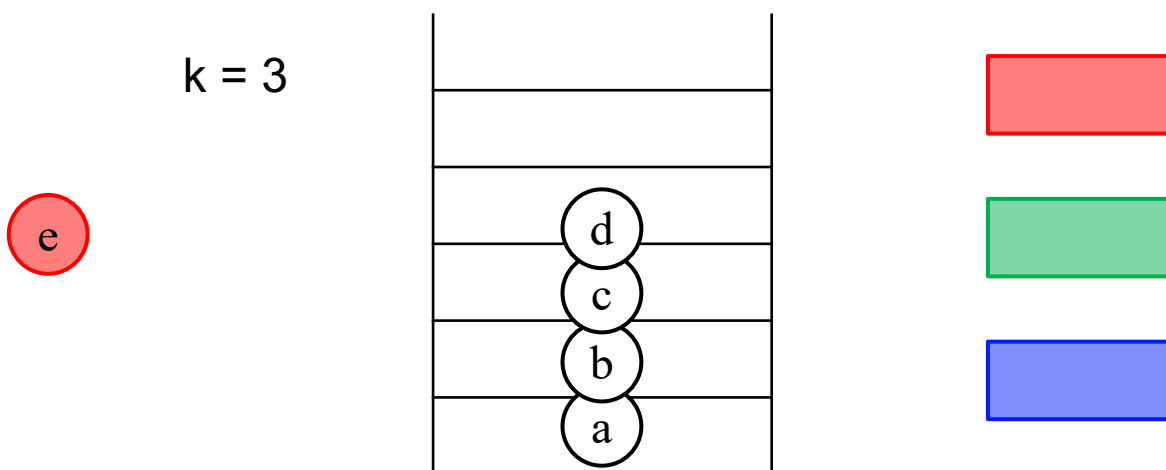
- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)

$k = 3$



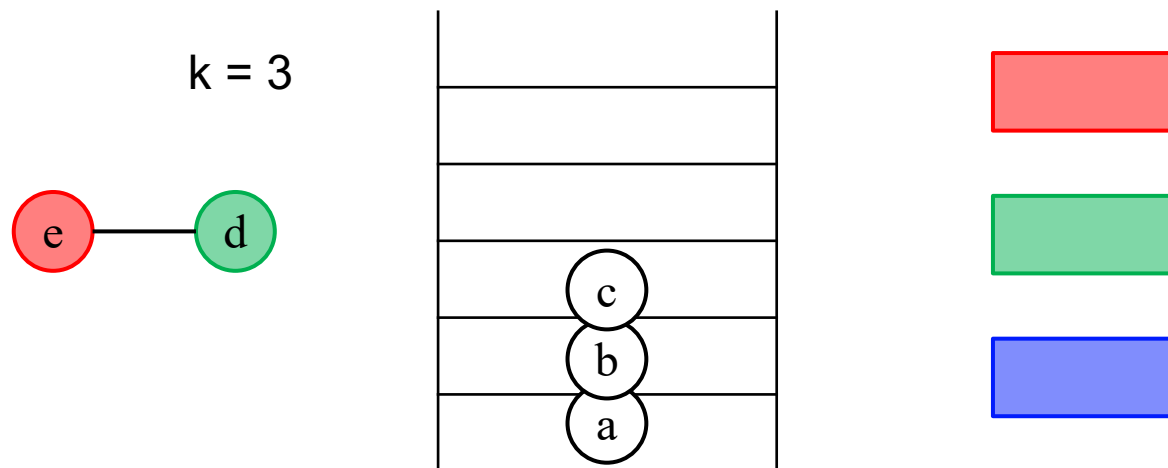
Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



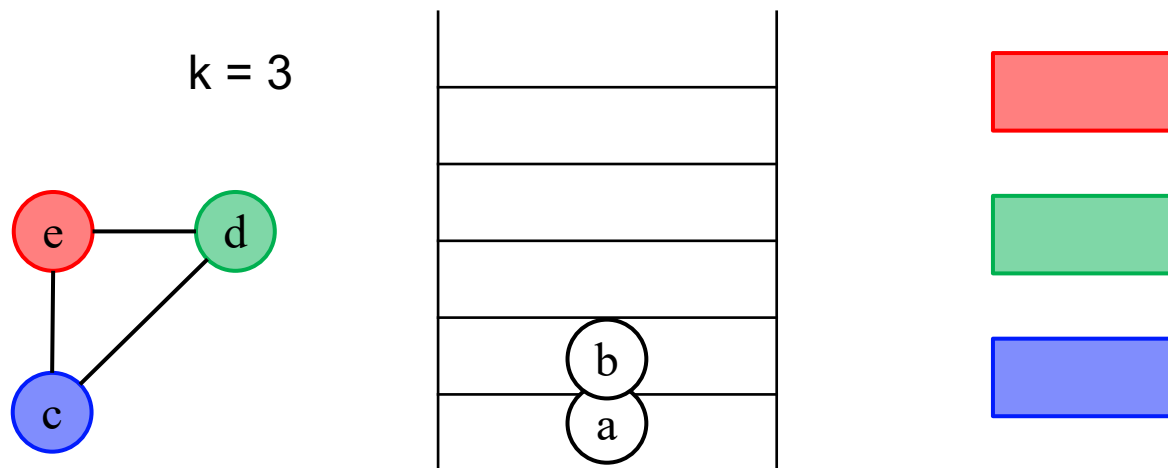
Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



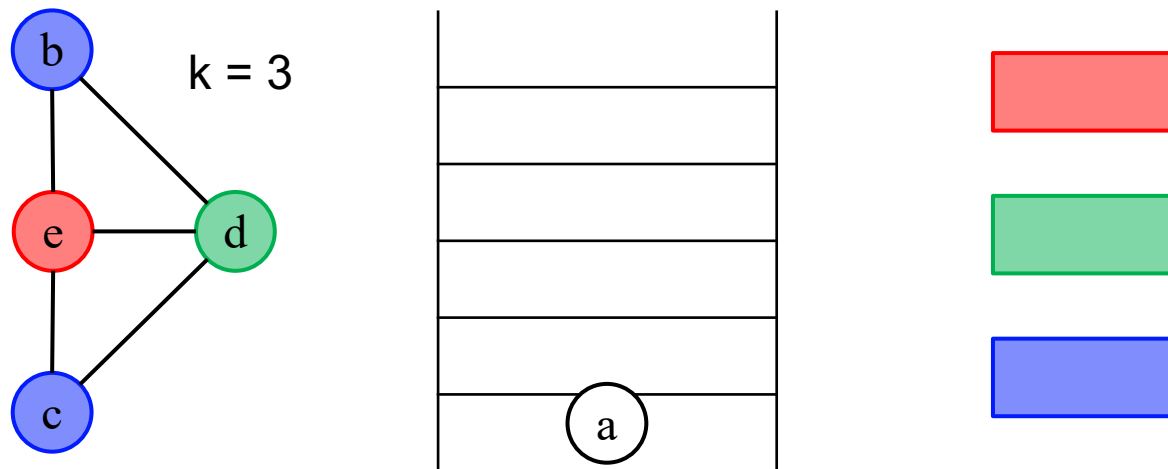
Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



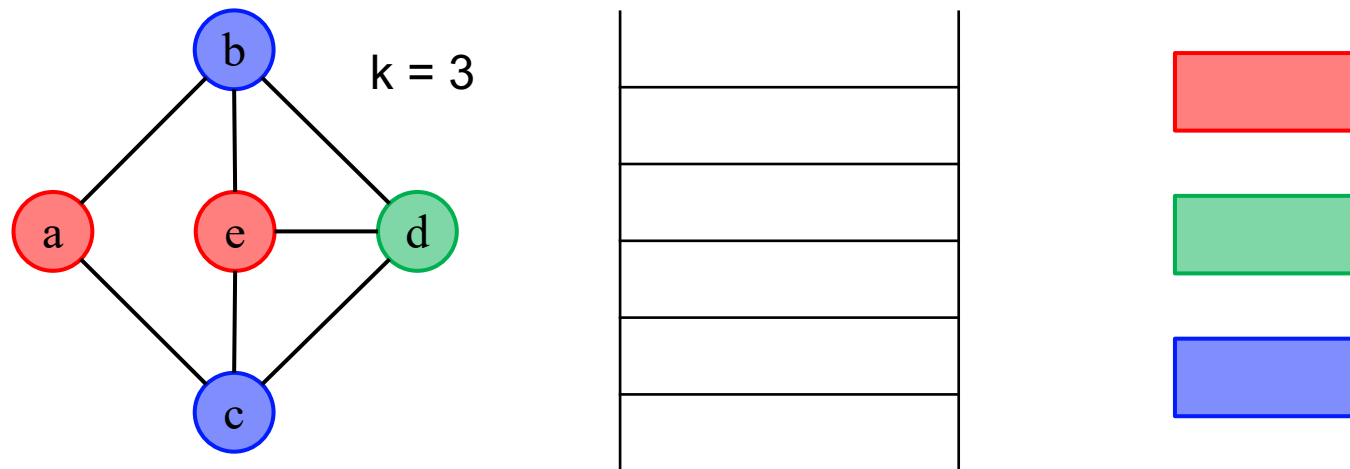
Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)



Chaitin's Algorithm

- Degree of a vertex is a loose upper bound on colorability
- A vertex whose degree $< k$ is always k -colorable
- Remove such vertices and push them to a stack until the graph becomes empty (color them later)

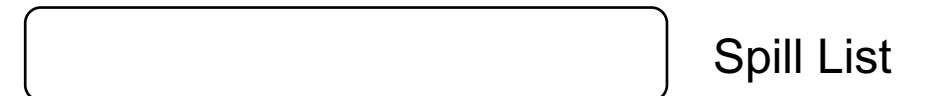
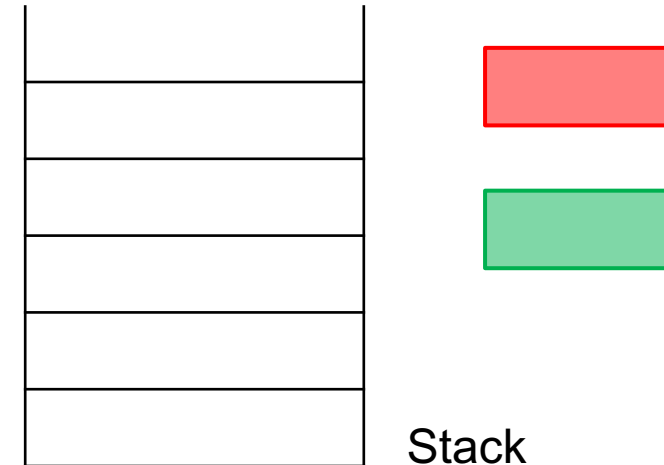
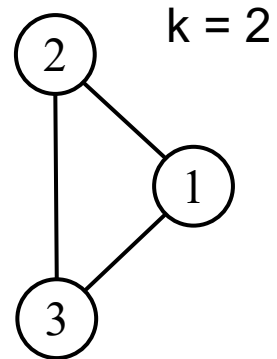


Chaitin's Algorithm

- When the algorithm reaches a state where every vertex has a degree $\geq k$
- Choose a vertex immediately to spill
 - put the vertex into a spill list
 - remove the vertex from the graph
 - continue moving vertices $< k$ degree from the graph to the stack
- If the spill list is not empty, insert spill code, rebuild conflict graph, and retry allocation.

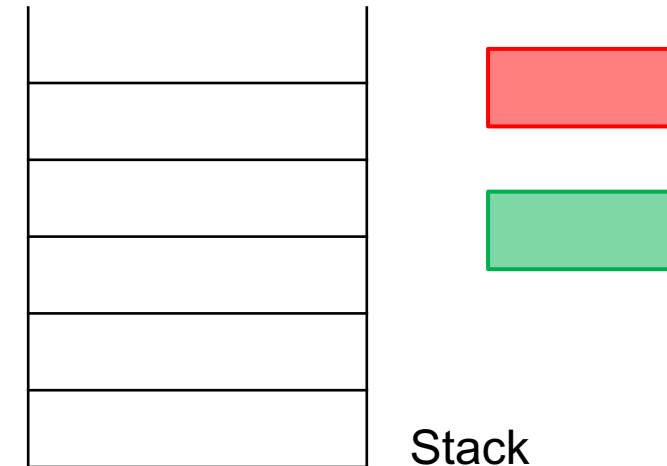
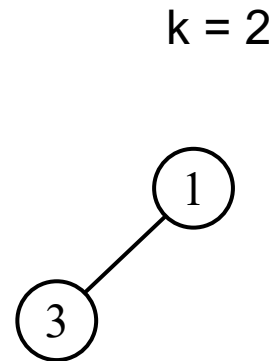
Chaitin's Algorithm

1.	LD	R1	#1028	// R1
2.	LD	R2	*R1	// R1 R2
3.	MUL	R3	R1 R2	// R1 R2 R3
4.	ST	x	R3	// R1 R2
5.	ST	y	R2	// R1
6.	ST	z	R1	//

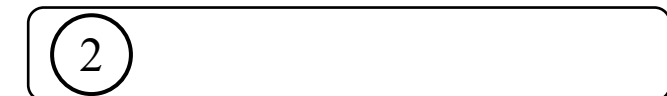


Chaitin's Algorithm

1.	LD	R1	#1028	// R1
2.	LD	R2	*R1	// R1 R2
3.	MUL	R3	R1 R2	// R1 R2 R3
4.	ST	x	R3	// R1 R2
5.	ST	y	R2	// R1
6.	ST	z	R1	//



Stack

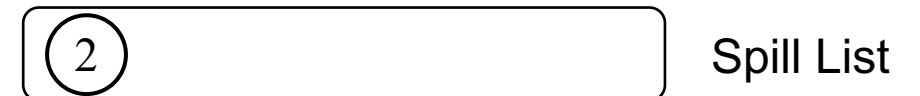
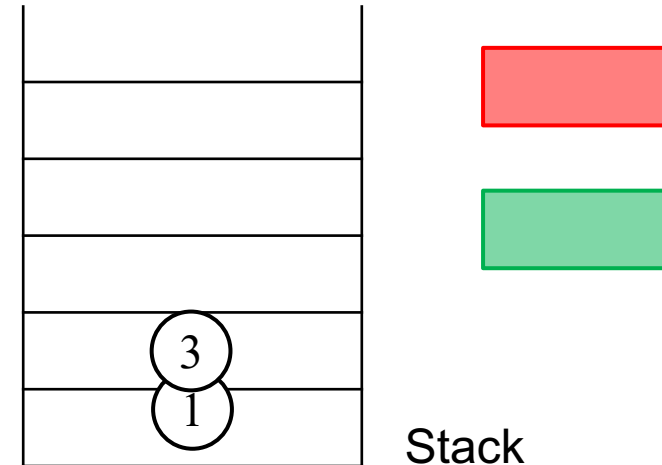


Spill List

Chaitin's Algorithm

1.	LD	R1	#1028	// R1
2.	LD	R2	*R1	// R1 R2
3.	MUL	R3	R1 R2	// R1 R2 R3
4.	ST	x	R3	// R1 R2
5.	ST	y	R2	// R1
6.	ST	z	R1	//

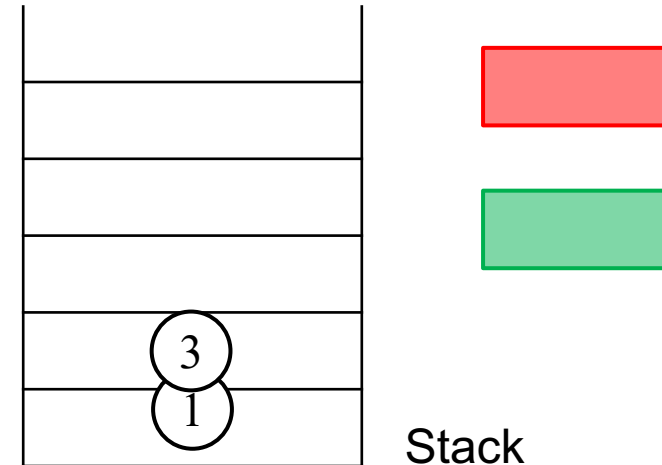
$k = 2$



Chaitin's Algorithm

1.	LD	R1	#1028	// R1
2.	LD	R2	*R1	// R1 R2
3.	MUL	R3	R1 R2	// R1 R2 R3
4.	ST	x	R3	// R1 R2
5.	ST	y	R2	// R1
6.	ST	z	R1	//

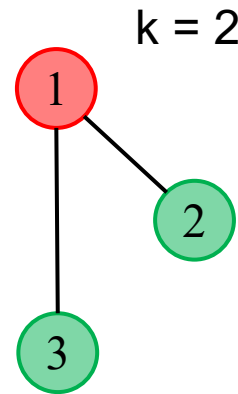
k = 2



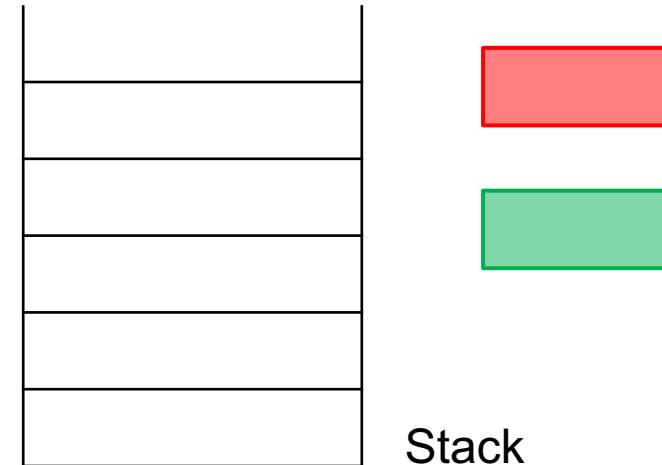
Spill R2!

Chaitin's Algorithm

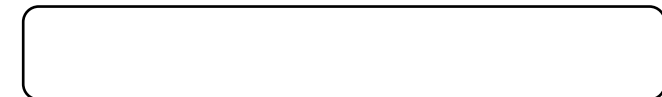
Replace R1 with R_{red}
Replace R2/R3 with R_{green}



1.	LD	R_{red}	#1028
2.	LD	R_{green}	* R_{red}
3.	ST	a	R_{green}
4.	MUL	R_{green}	R_{red} R_{red}
5.	ST	x	R_{green}
6.	LD	R_{green}	a
7.	ST	y	R_{green}
8.	ST	z	R_{red}



Stack

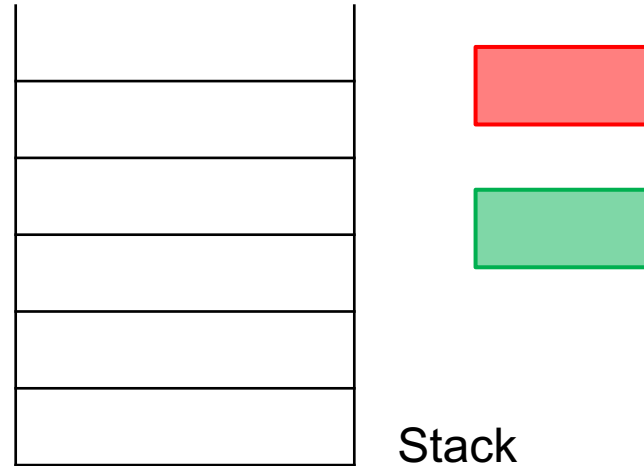
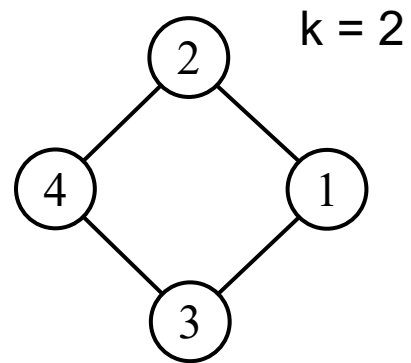


Spill List

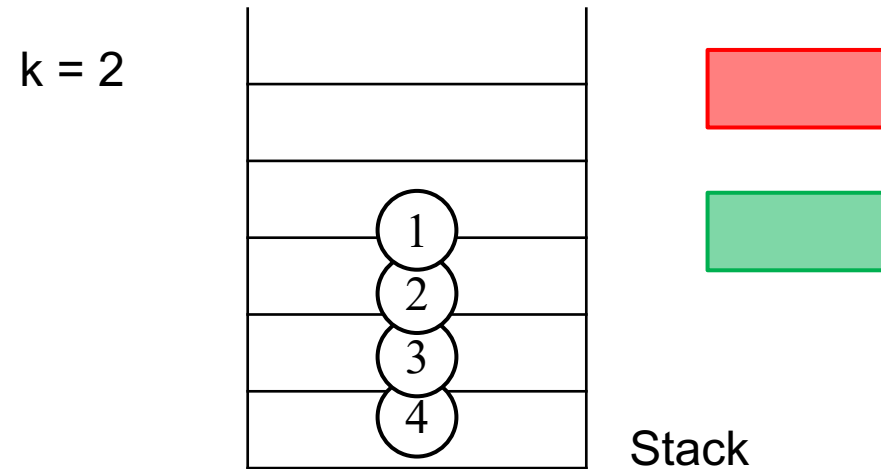
Chaitin-Briggs Algorithm

- When Chaitin's algorithm reaches a state where every vertex has a degree $\geq k$
- Choose a vertex immediately to spill
 - ~~put the vertex into a spill list~~
 - ~~...~~
- **Briggs:**
 - Push the vertex (to spill) to the stack.
 - We may have a color available when it is popped.

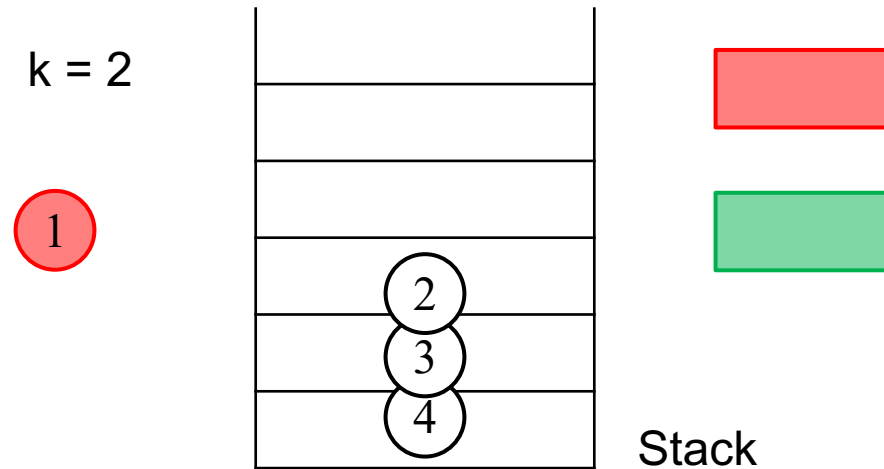
Chaitin-Briggs Algorithm



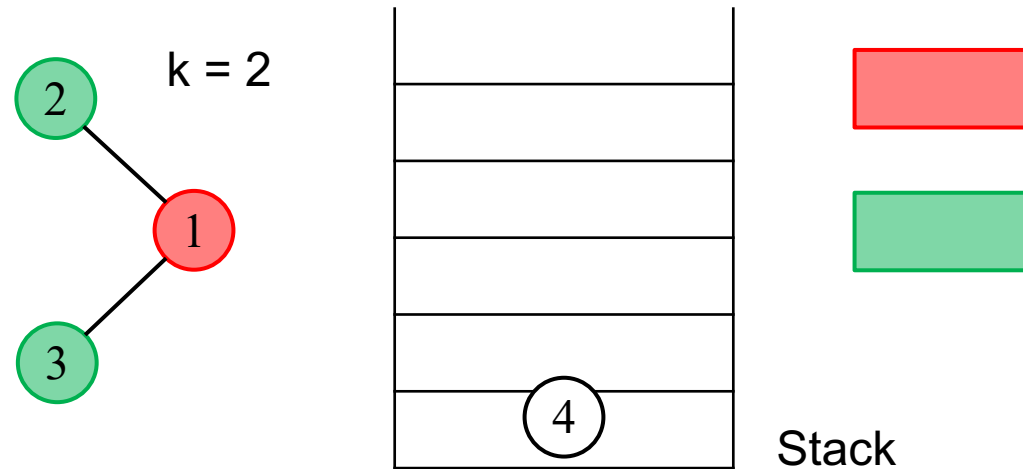
Chaitin-Briggs Algorithm



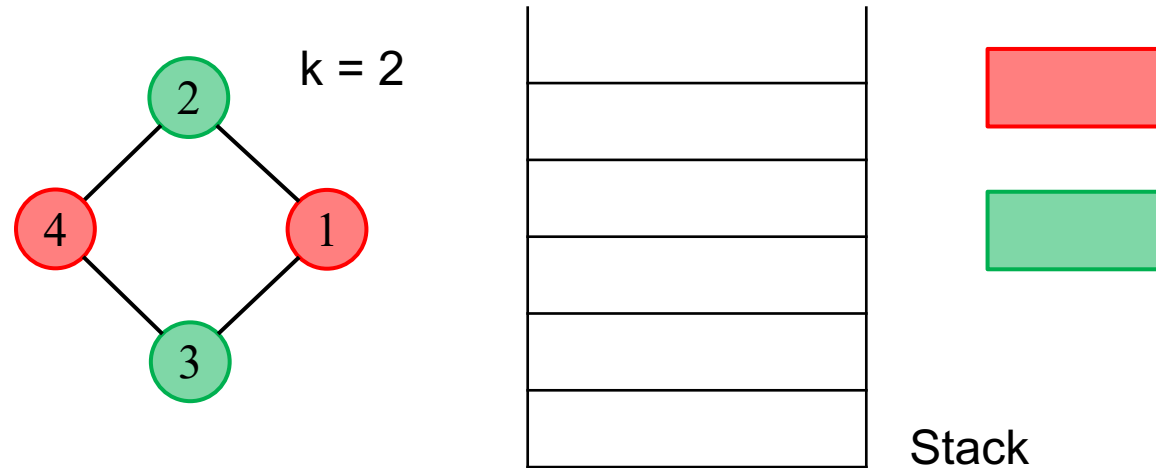
Chaitin-Briggs Algorithm



Chaitin-Briggs Algorithm



Chaitin-Briggs Algorithm



Chaitin-Briggs Algorithm

- Compared to Chaitin's algorithm, Chitin-Briggs algorithm delays the spill operation
 - saves a spill list
 - reduces spill code

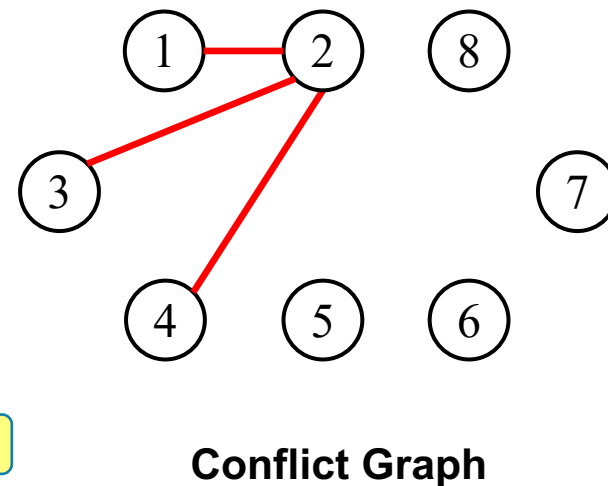
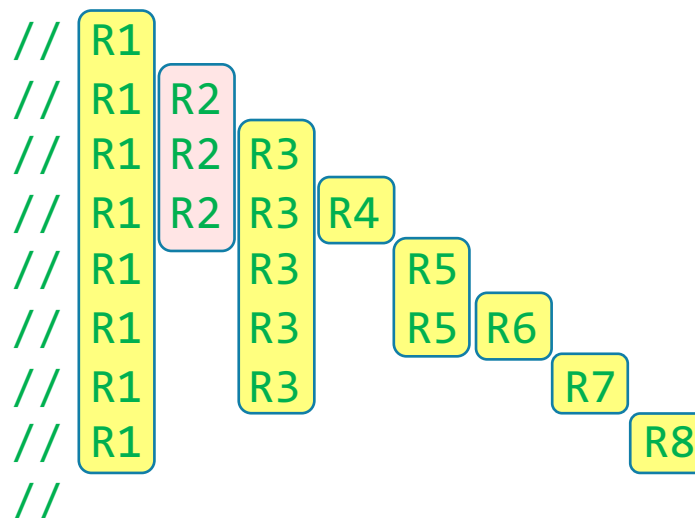
Spill Candidates

- Minimize spill cost, *i.e.*, the loads/stores needed
 - *e.g.*, avoid inner loops
- The higher the degree of a vertex
 - The greater the chance that spilling it will help coloring
- Don't spill a value which is defined immediately followed by use
 - Spilling does not decrease live range

Alternative Spilling

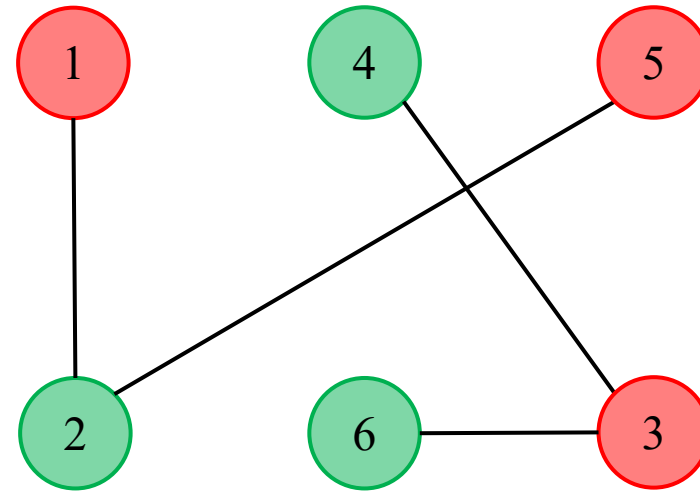
- Splitting Live Ranges, Coalescing Virtual Registers, etc.
- Recall the **conflict graph**:

1. LD	R1	#1028	
2. LD	R2	*R1	
3. MUL	R3	R1	R2
4. LD	R4	x	
5. SUB	R5	R4	R2
6. LD	R6	z	
7. MUL	R7	R5	R6
8. SUB	R8	R7	R3
9. ST	*R1	R8	



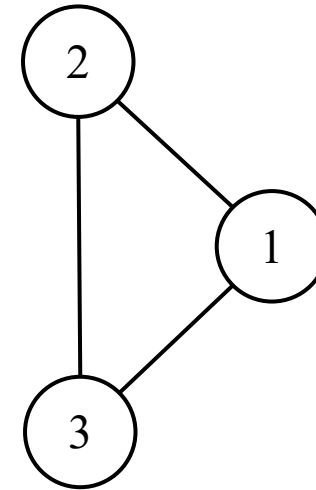
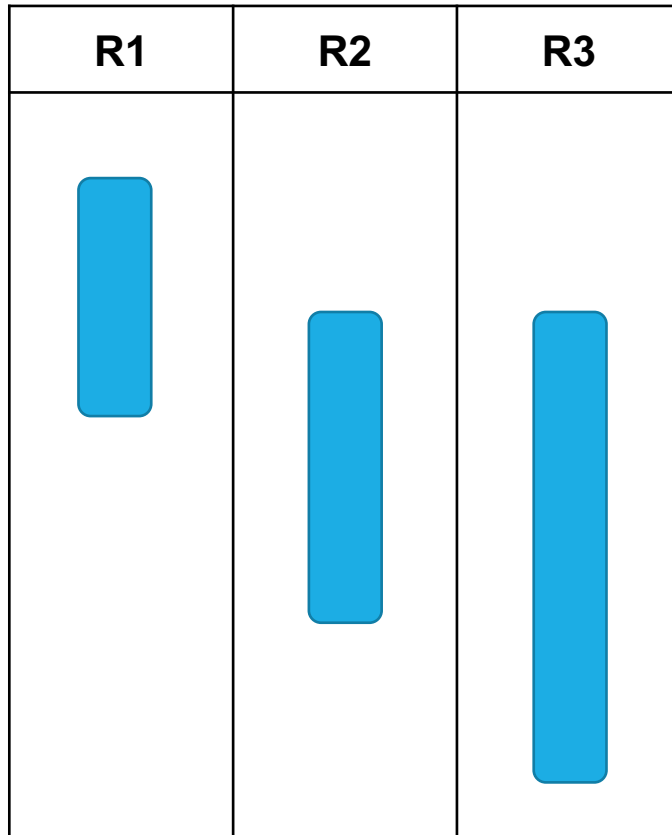
Splitting Live Range

R_1 R_4 R_5	R_2 R_6	R_3

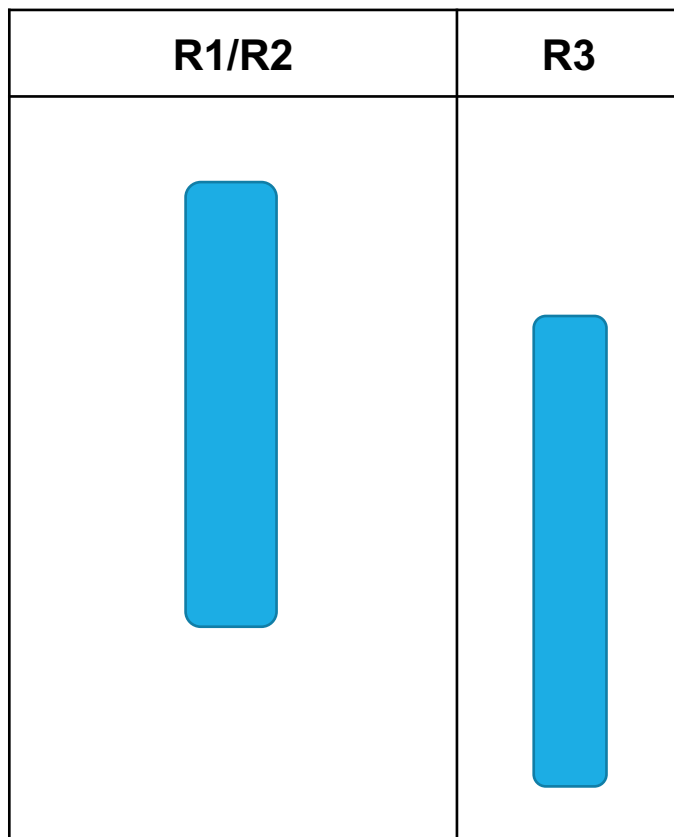


The conflict graph is 2-colorable!

Coalescing Virtual Registers



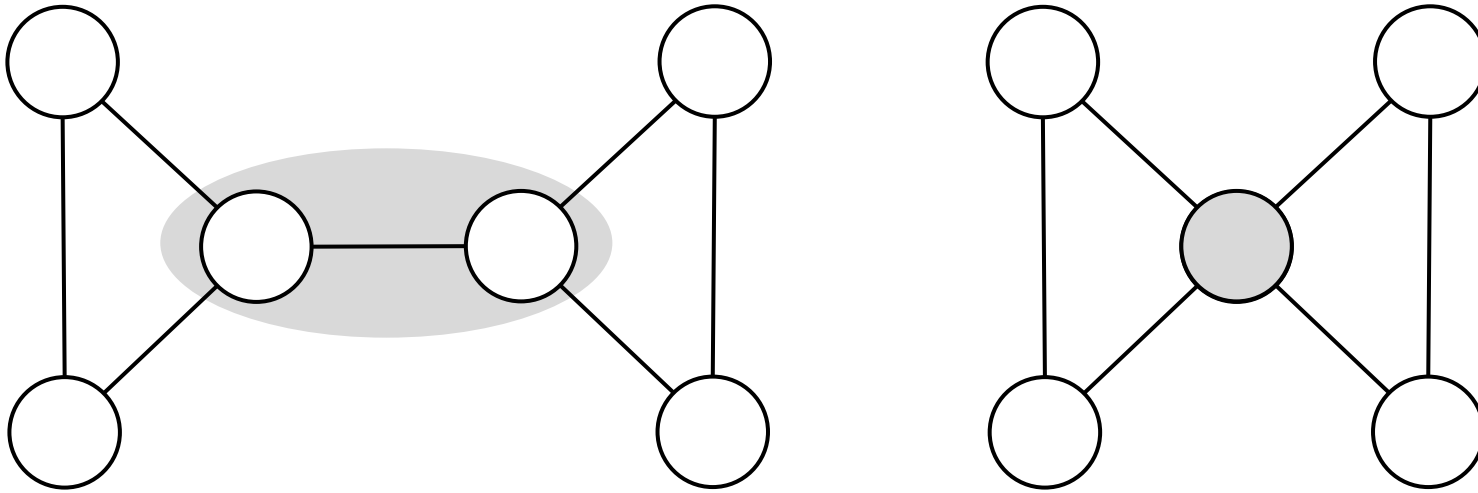
Coalescing Virtual Registers



Assume $R1 = R2$ by `LD R2 R1`



Coalescing Doesn't Always Work

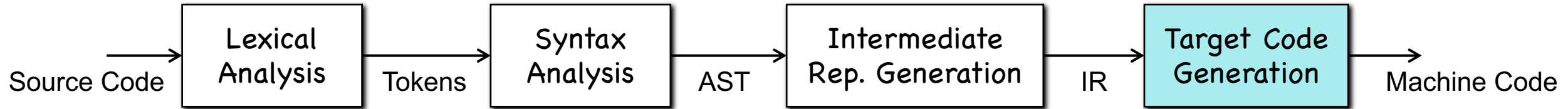


A vertex such that its degree $< k$ is always k -colorable



Degree: $3 \rightarrow 4$
Harder to color

Summary



- Speed: Registers > Memory
- Physical machines have limited number of registers
- **Register allocation:** ∞ virtual registers $\rightarrow$ k physical registers
 - Part I – Local Register Allocation
 - Part II – Global Register Allocation

Chapter 10

Instruction Scheduling

PART I: Scheduling Constraints

Performance is Order Dependent

• $a = 2 * a * b * c$

LD	R1,	a
ADD	R1,	R1, R1
LD	R2,	b
MUL	R1,	R1, R2
LD	R2,	c
MUL	R1,	R1, R2
ST	a,	R1

Scheduling Constraints

- Storage (Fake) Dependence --- Rename to Remove
 - Antidependence & **Output dependence (Write-After-Write)**

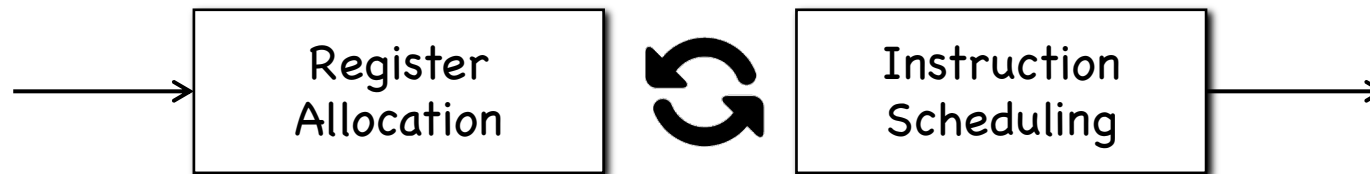
LD	R1,	a
ADD	R1,	R1, R1
LD	<u>R2</u> ,	b
MUL	R1,	R1, R2
LD	<u>R2</u> ,	c
MUL	R1,	R1, R2
ST	a,	R1

LD	R1,	a
LD	<u>R2</u> ,	b
LD	<u>R3</u> ,	c
ADD	R1,	R1, R1
MUL	R1,	R1, R2
MUL	R1,	R1, R3
ST	a,	R1



Register Allocation vs. Scheduling

- Which should we do first?
- **Register Allocation:** Designed to use fewer registers
- **Instruction Scheduling:** Prefer more registers



- It's a tough question! It depends on applications
- Multi-objective optimization https://en.wikipedia.org/wiki/Multi-objective_optimization

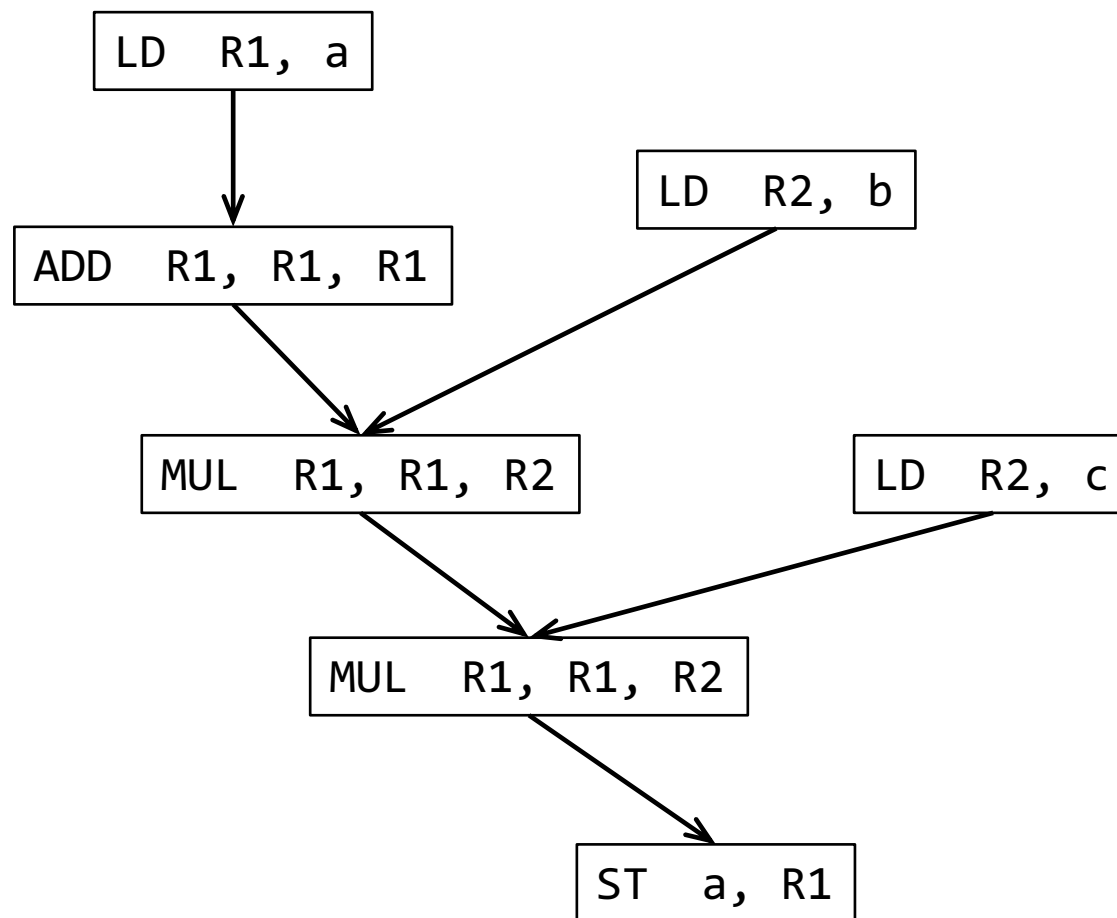
PART II: Basic Block Scheduling

Dependence Graph

- Node: Instructions
- Edge: True Dependence (**Read-After-Write Dependence**)

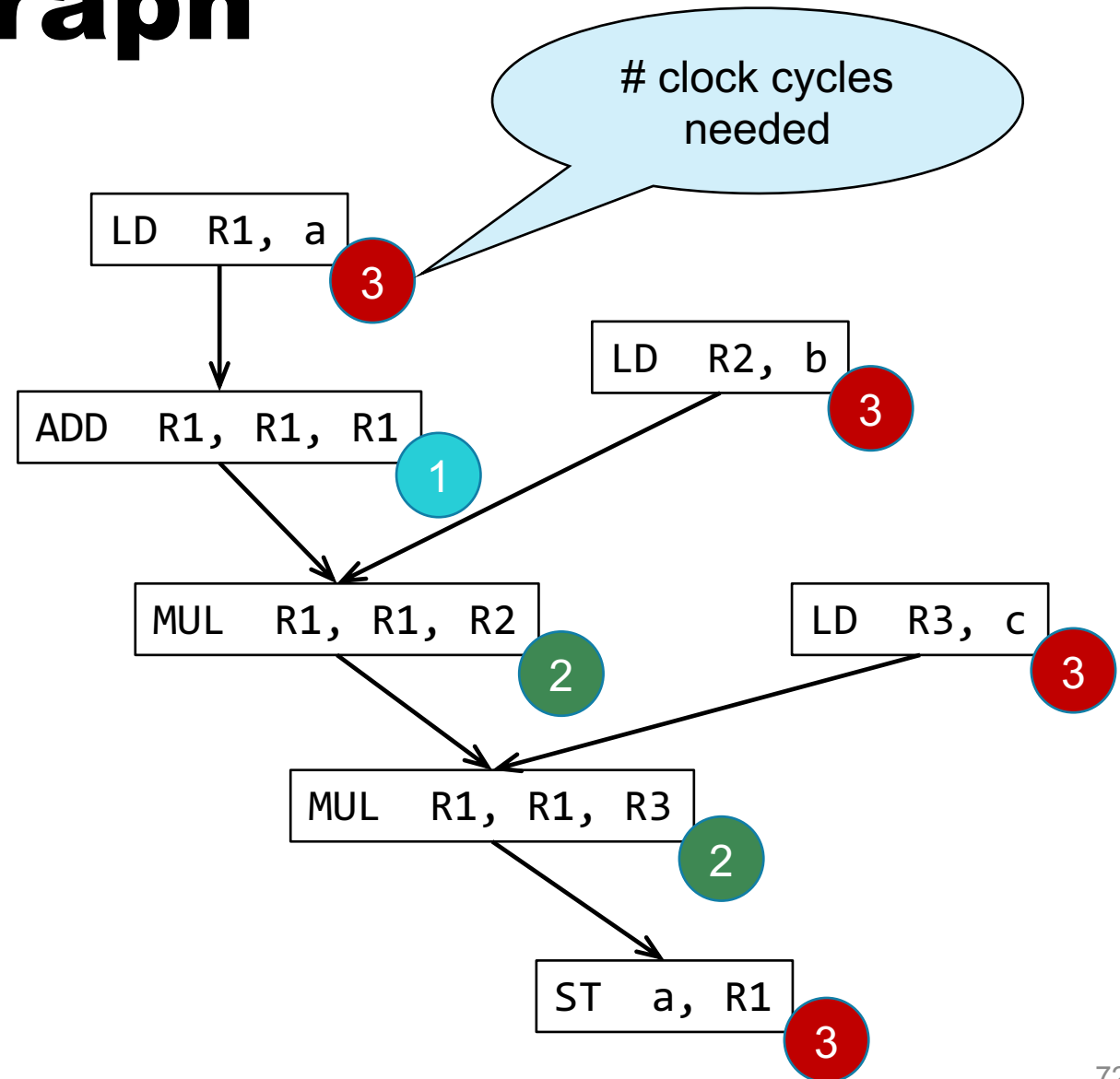
Dependence Graph

LD	R1,	a
ADD	R1,	R1, R1
LD	R2,	b
MUL	R1,	R1, R2
LD	R2,	c
MUL	R1,	R1, R2
ST	a,	R1



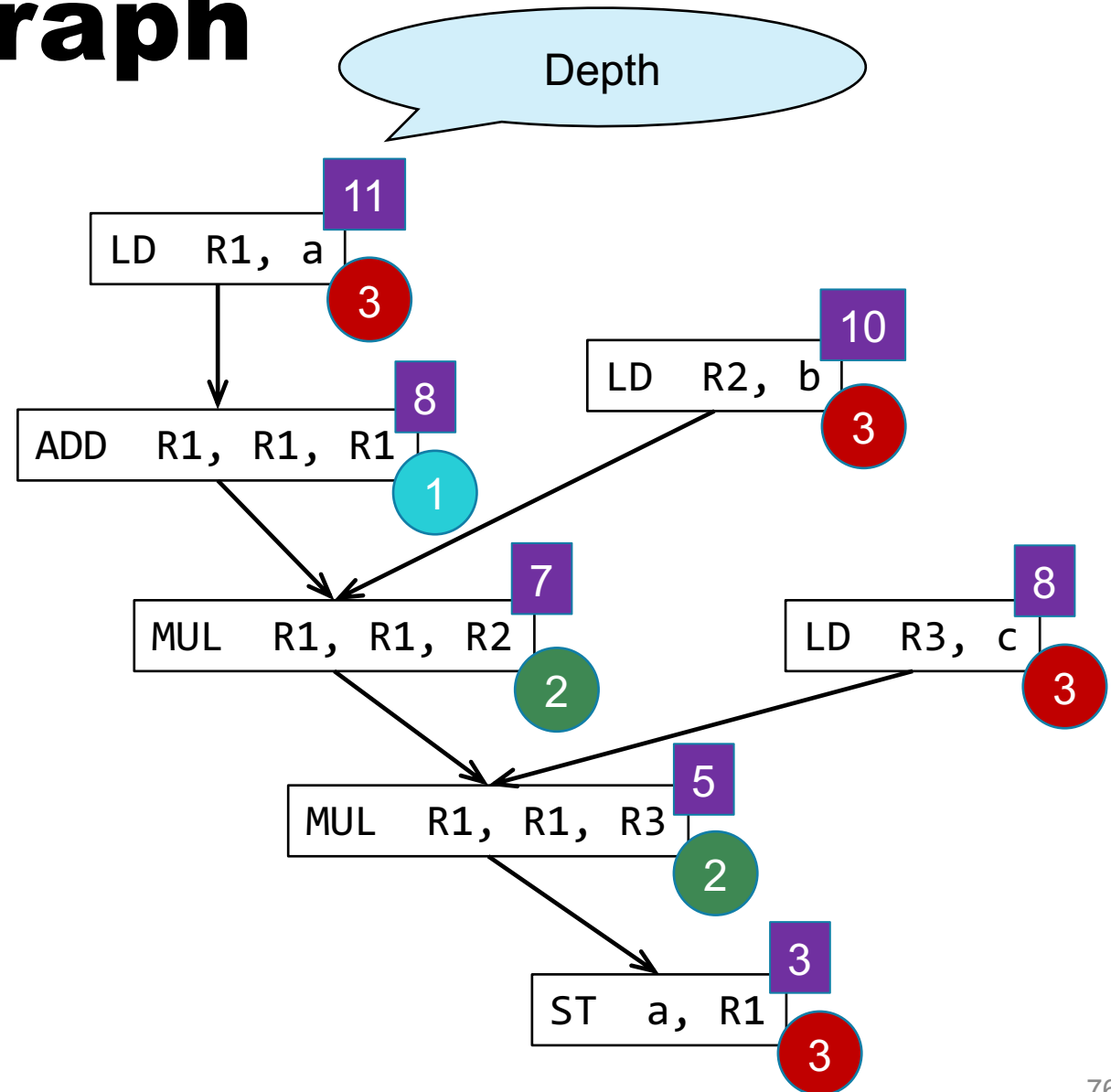
Dependence Graph

LD	R1,	a
ADD	R1,	R1, R1
LD	R2,	b
MUL	R1,	R1, R2
LD	R3,	c
MUL	R1,	R1, R3
ST	a,	R1



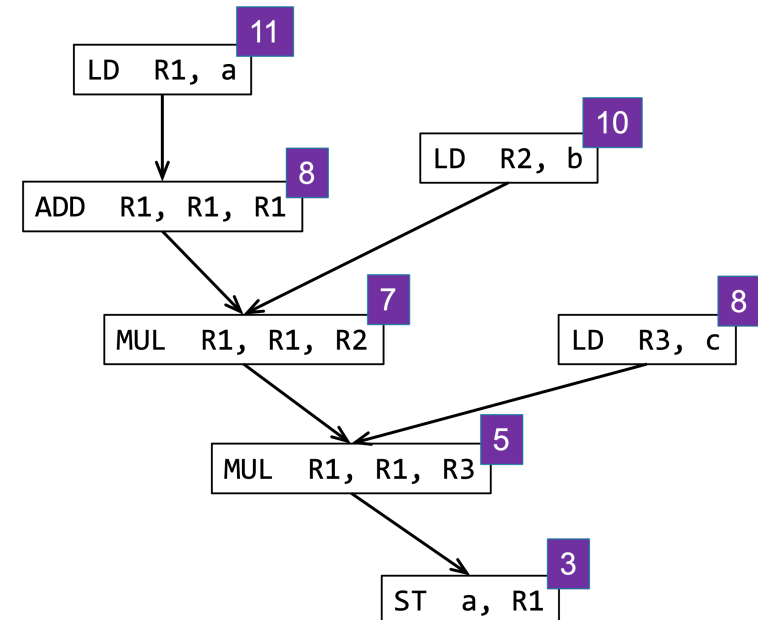
Dependence Graph

LD	R1,	a
ADD	R1,	R1, R1
LD	R2,	b
MUL	R1,	R1, R2
LD	R3,	c
MUL	R1,	R1, R3
ST	a,	R1



List Scheduling

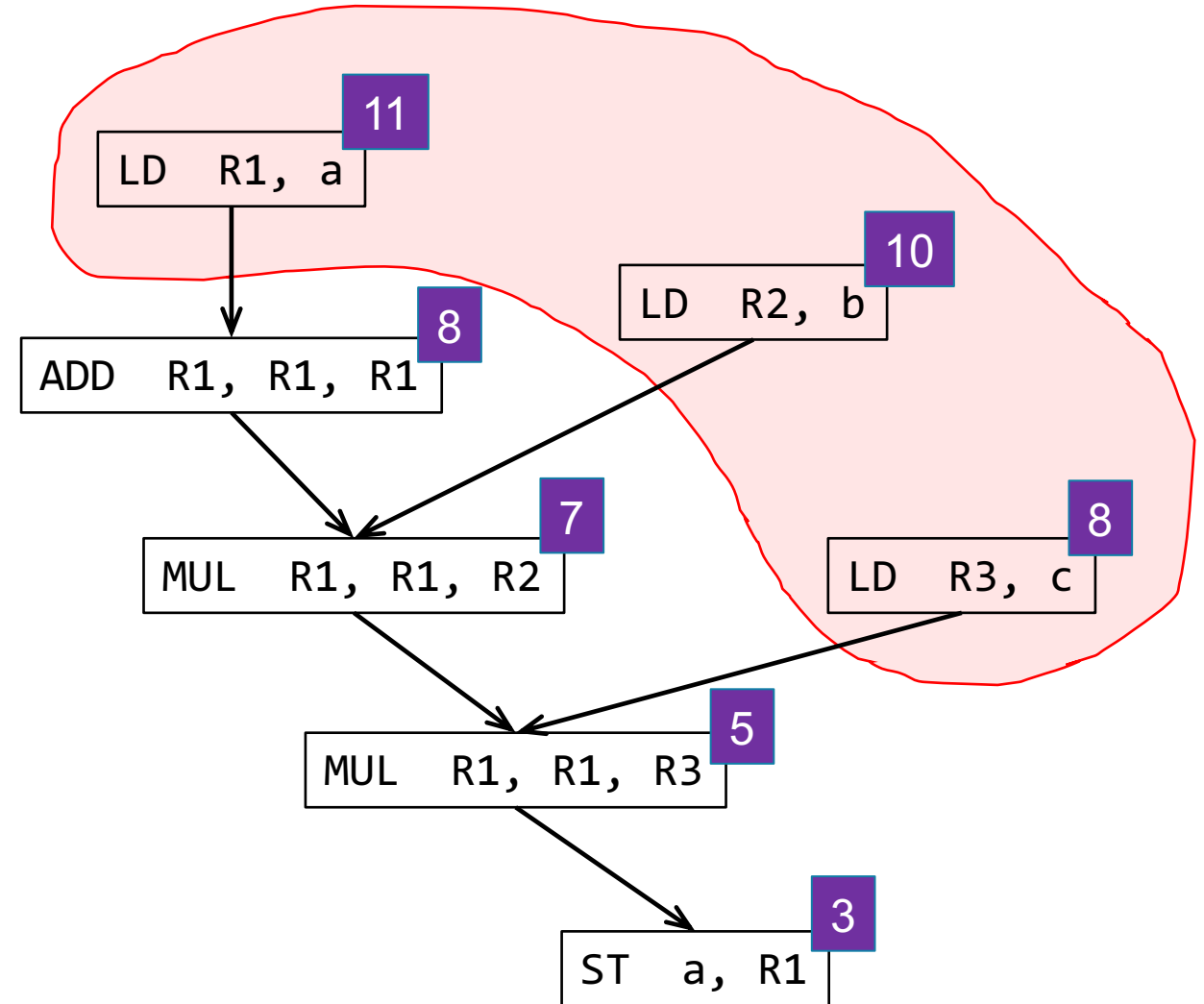
- Check each clock cycle
- Schedule instructions when they are ready
- When multi instructions can be scheduled, check their **priority**
 - The longest latency path (max depth)
 - Using the most resources
 - ...



List Scheduling

- Cycle = 1

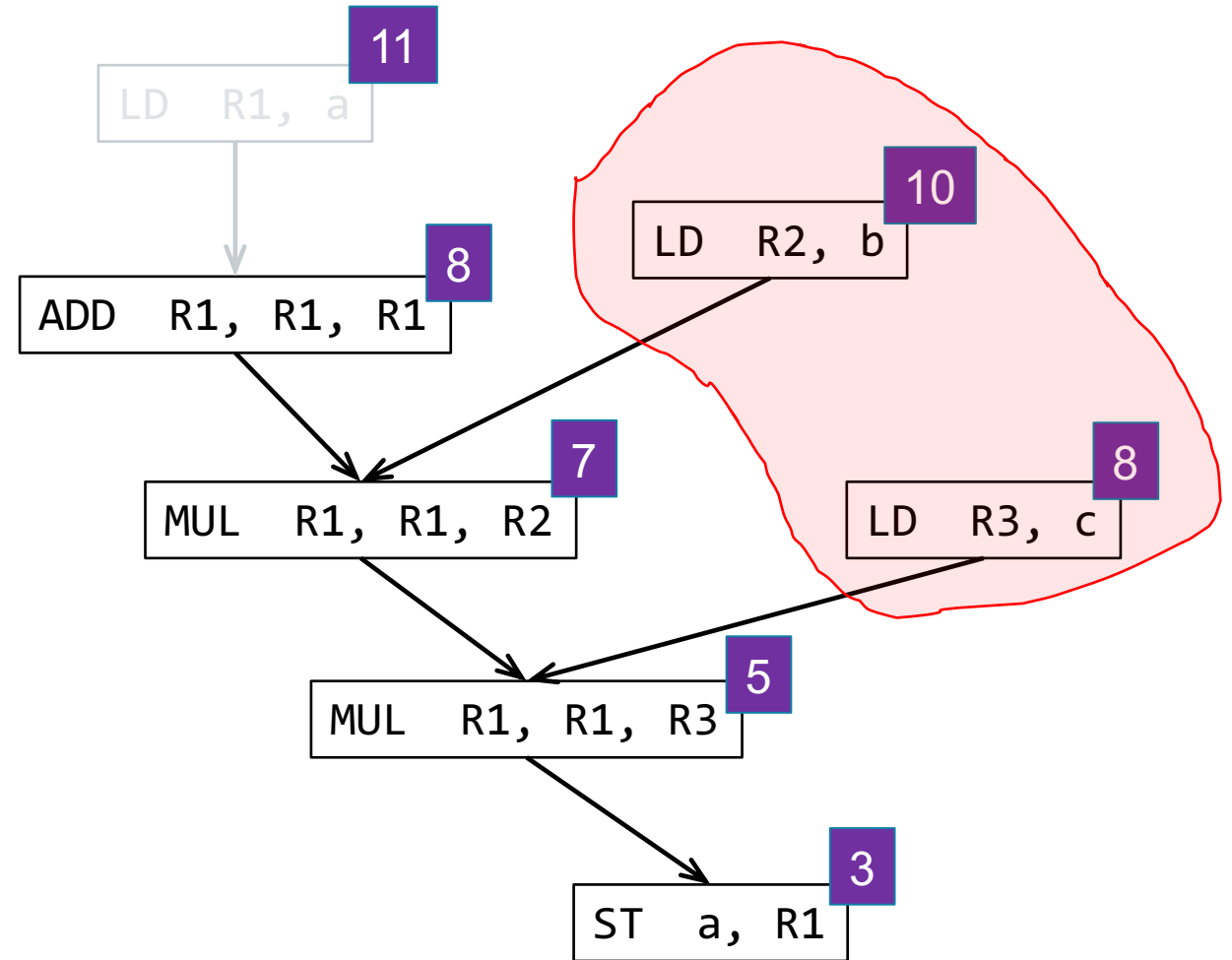
LD R1, a



List Scheduling

- Cycle = 2

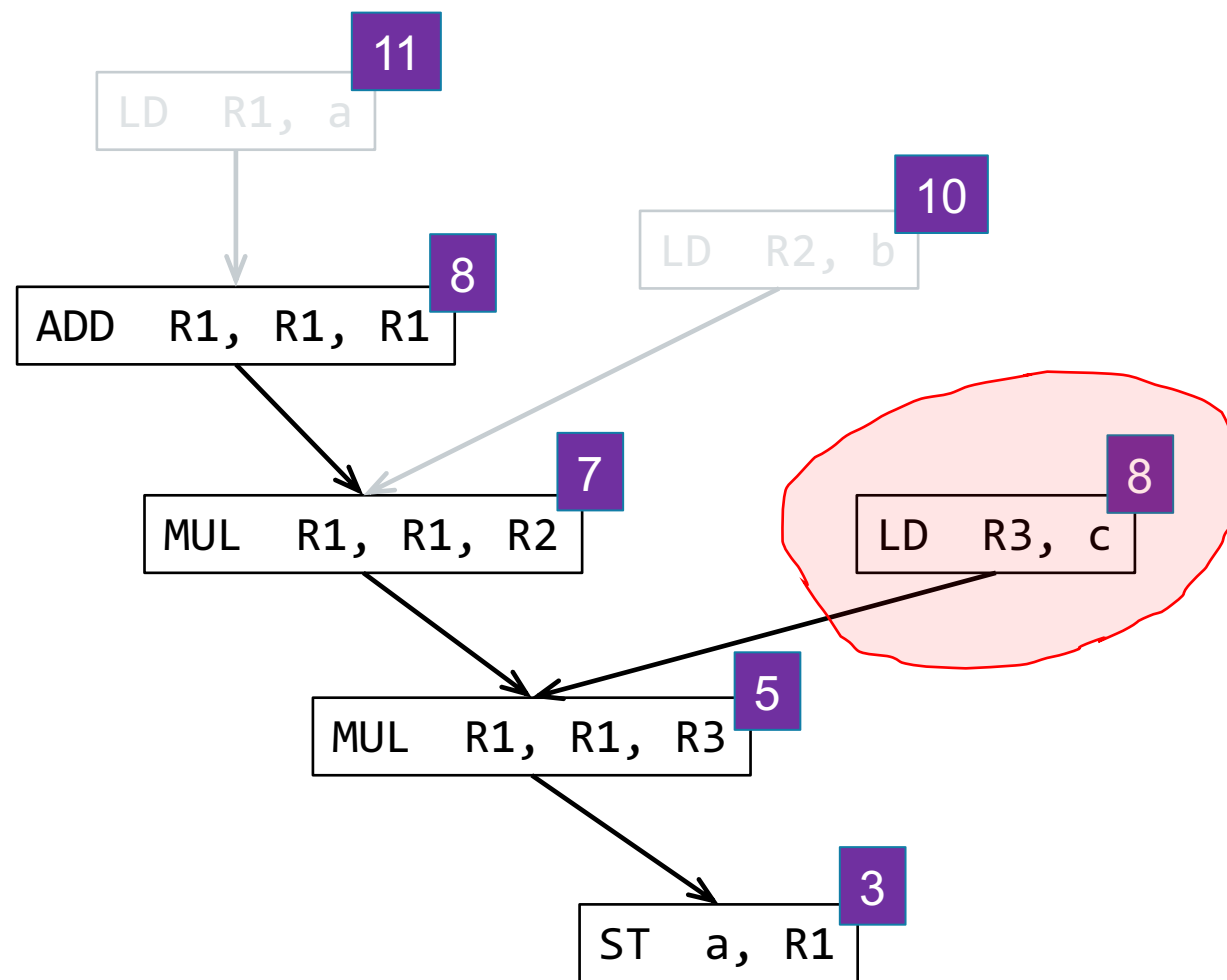
LD	R1,	a
LD	R2,	b



List Scheduling

- Cycle = 3

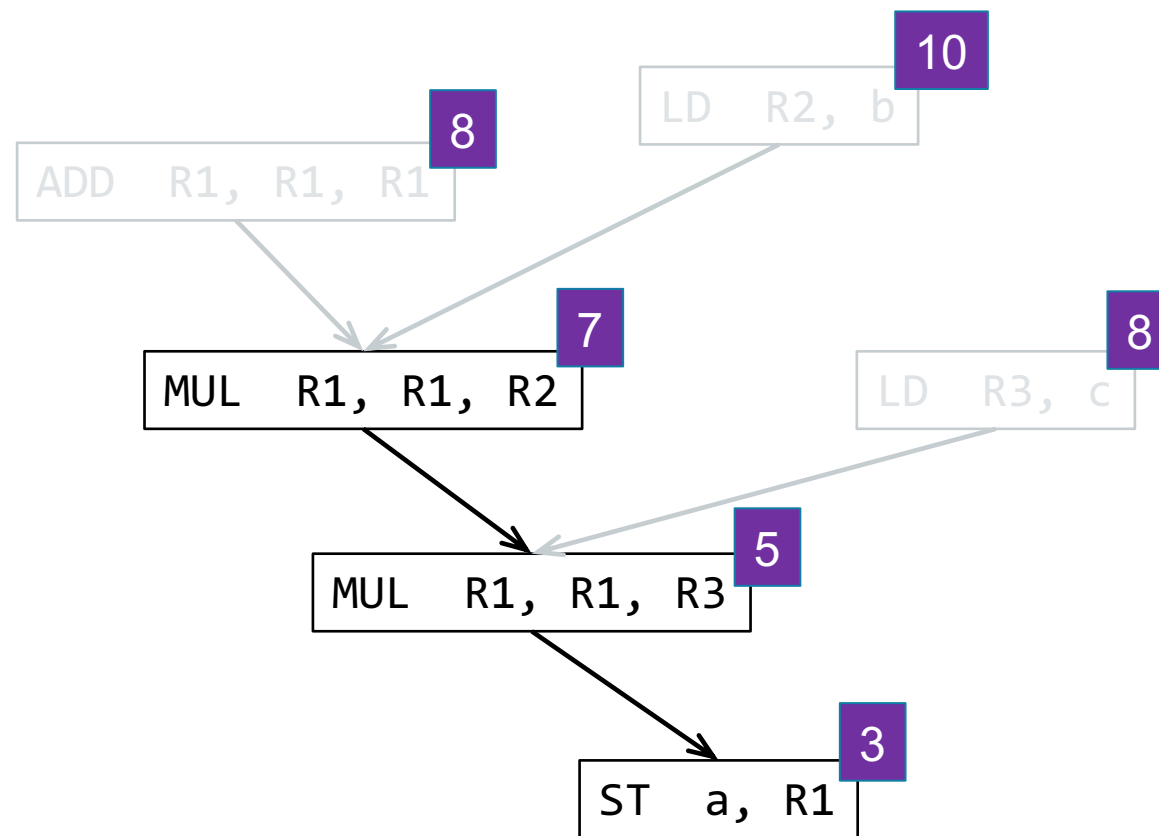
LD	R1,	a
LD	R2,	b
LD	R3,	c



List Scheduling

- Cycle = 4

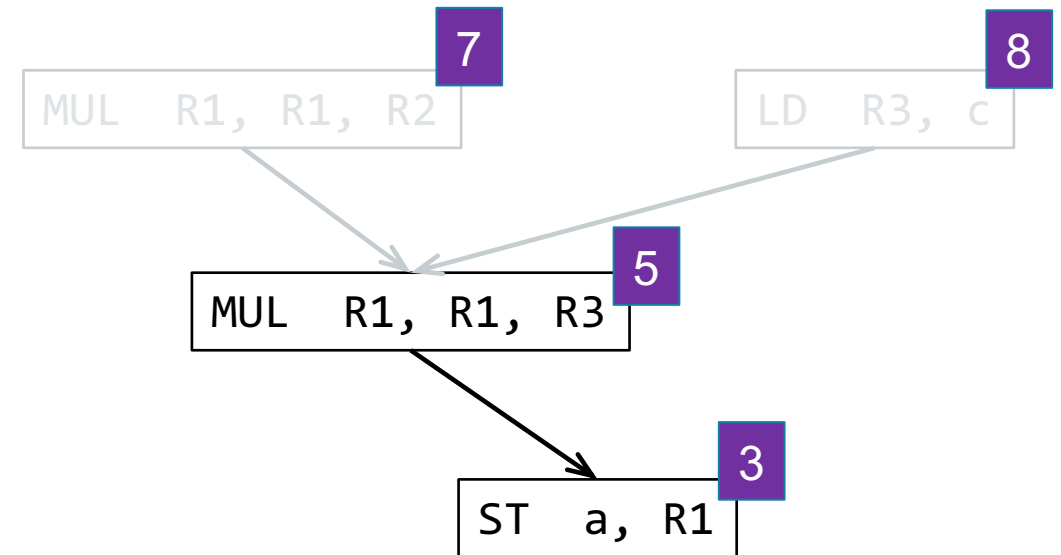
LD	R1,	a
LD	R2,	b
LD	R3,	c
ADD	R1,	R1, R1



List Scheduling

- Cycle = 5

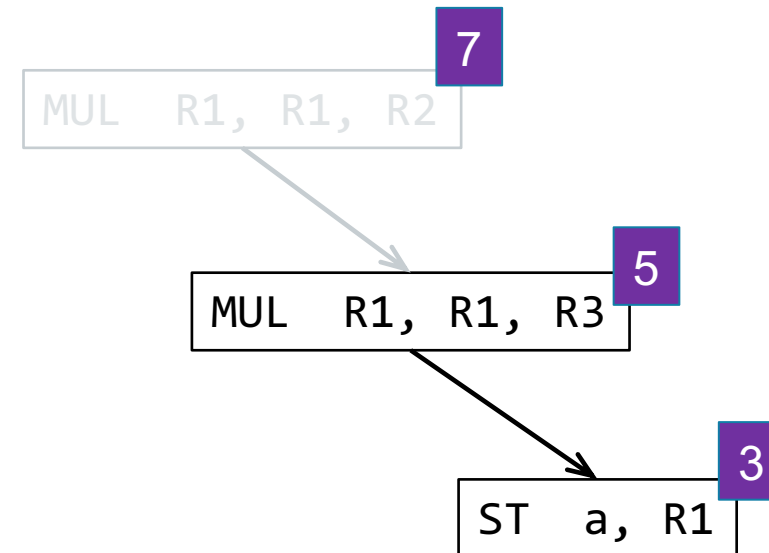
LD	R1,	a
LD	R2,	b
LD	R3,	c
ADD	R1,	R1, R1
MUL	R1,	R1, R2



List Scheduling

- Cycle = 6

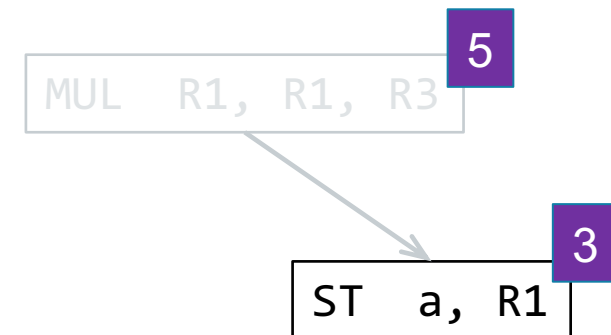
LD	R1,	a
LD	R2,	b
LD	R3,	c
ADD	R1,	R1, R1
MUL	R1,	R1, R2
NOP	(no operation)	



List Scheduling

- Cycle = 7, 8

LD	R1,	a
LD	R2,	b
LD	R3,	c
ADD	R1,	R1, R1
MUL	R1,	R1, R2
NOP (no operation)		
MUL	R1,	R1, R3
NOP (no operation)		



List Scheduling

- Cycle = 9, 10, 11

LD	R1,	a
LD	R2,	b
LD	R3,	c
ADD	R1,	R1, R1
MUL	R1,	R1, R2
NOP (no operation)		
MUL	R1,	R1, R3
NOP (no operation)		
ST	a,	R1

ST a, R1 3

List Scheduling

• $a = 2 * a * b * c$

List Scheduling



LD	R1,	a
ADD	R1,	R1, R1
LD	R2,	b
MUL	R1,	R1, R2
LD	R2,	c
MUL	R1,	R1, R2
ST	a,	R1

LD	R1,	a
LD	R2,	b
LD	R3,	c
ADD	R1,	R1, R1
MUL	R1,	R1, R2
MUL	R1,	R1, R3
ST	a,	R1

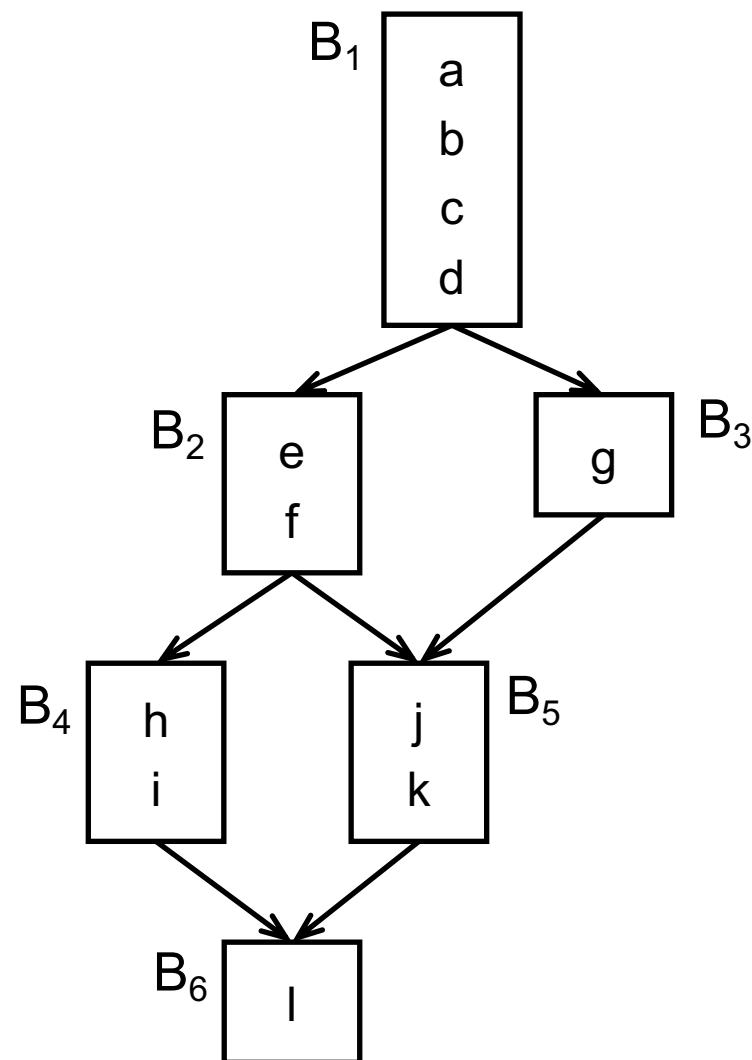
Summary of List Scheduling

- Check each clock cycle
- Schedule instructions when they are ready
- When multiple instructions can be scheduled, check their **priority**
 - The longest latency path (max depth)
 - Using the most resources
 - ...

PART III: Global Code Scheduling

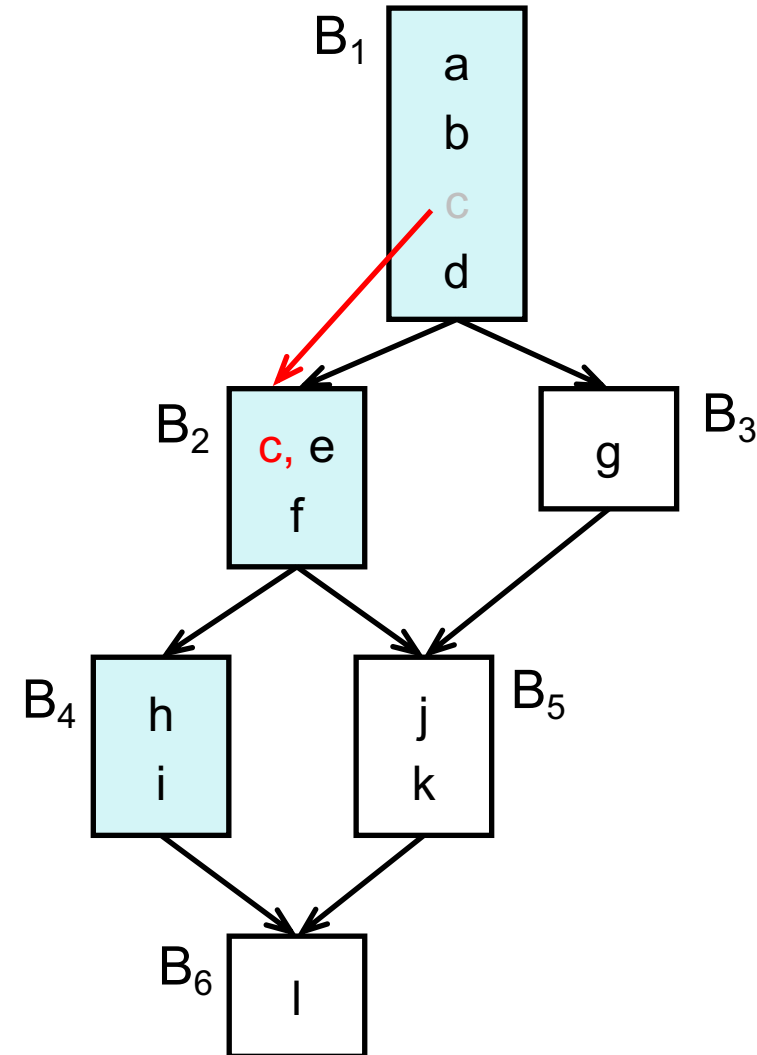
Extended Basic Block (EBB)

- EBB is a maximal set of blocks such that
- (1) It has a single entry;
- (2) All blocks except for the entry has only one predecessor.



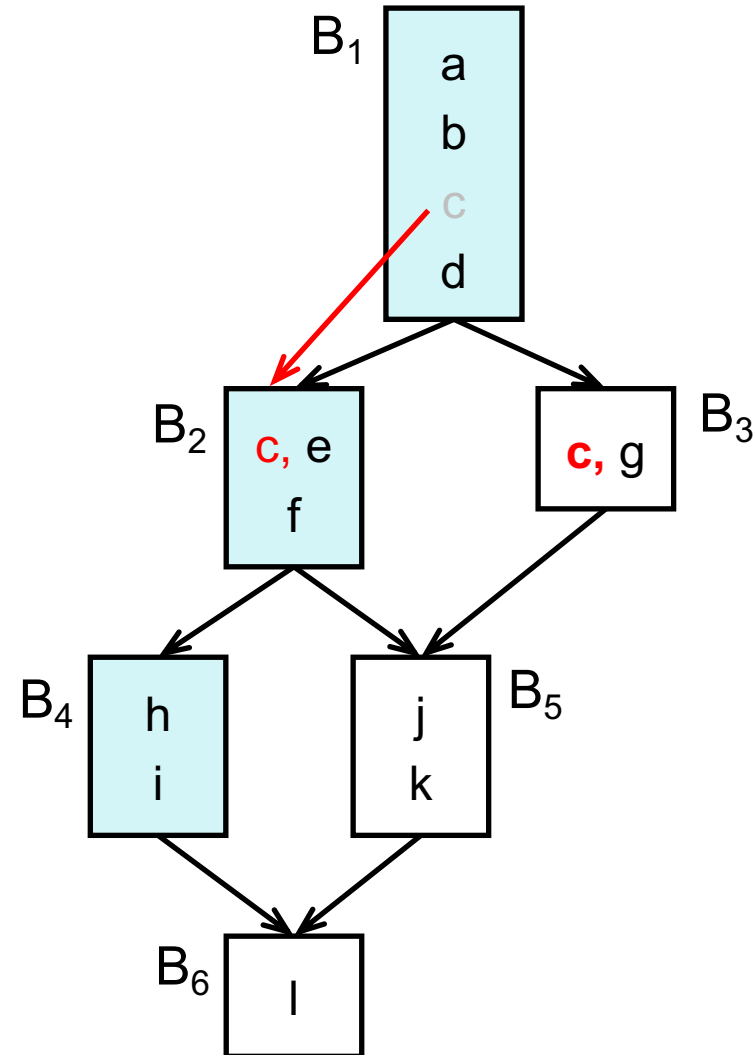
Global Scheduling

- Schedule each EBB path like a basic block, e.g., via list scheduling
- Scheduling may change the order of instructions
 - **Downward code motion**
 - **Upward code motion**



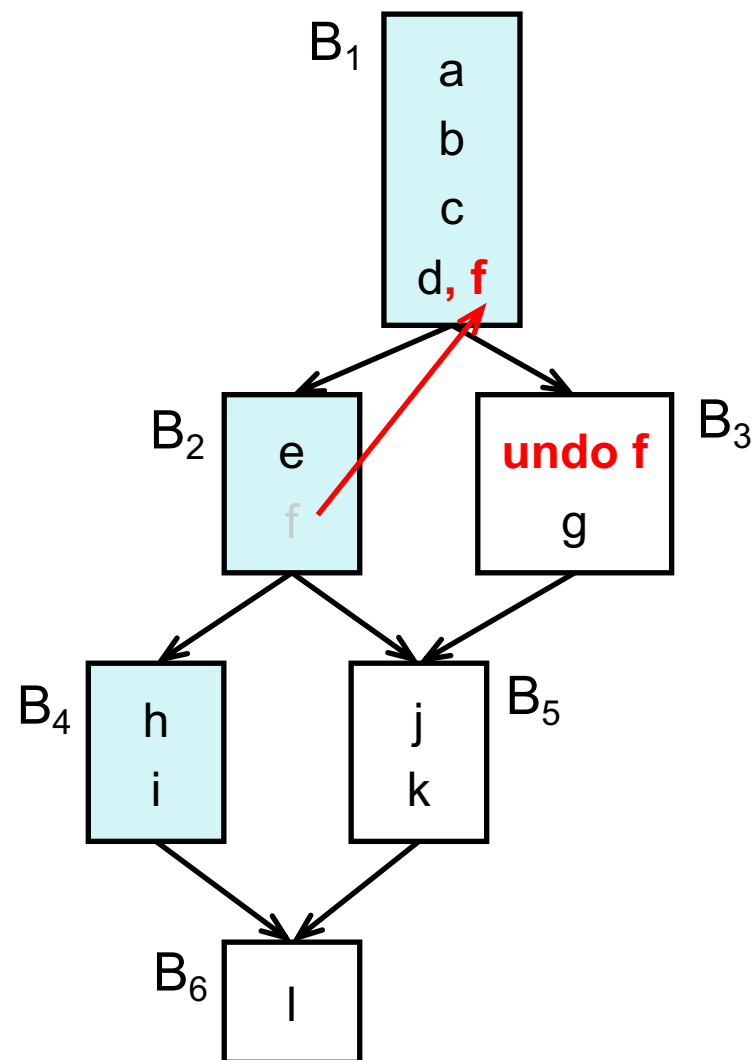
Downward Code Motion

- Schedule each EBB path like a basic block, e.g., via list scheduling
- Scheduling may change the order of instructions
- Extra code may need to be inserted



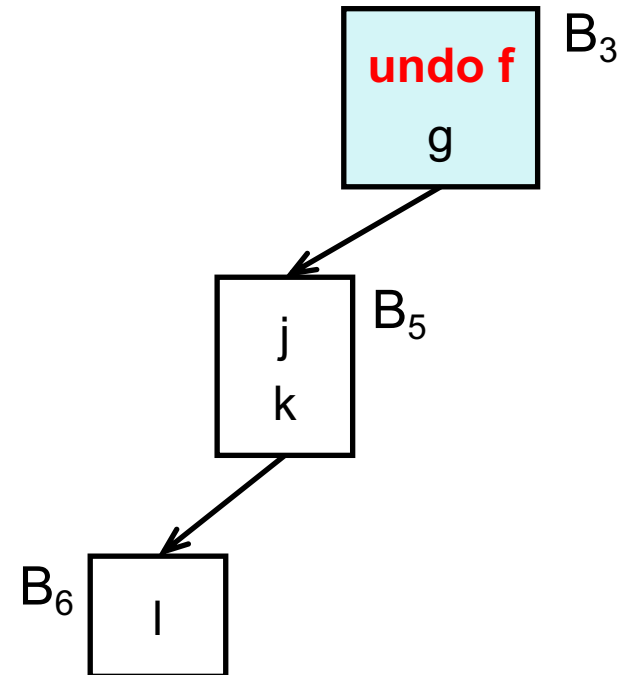
Upward Code Motion

- Schedule an EBB path like a basic block, e.g., via list scheduling.
- Scheduling may change the order of instructions
- Extra code may need to be inserted



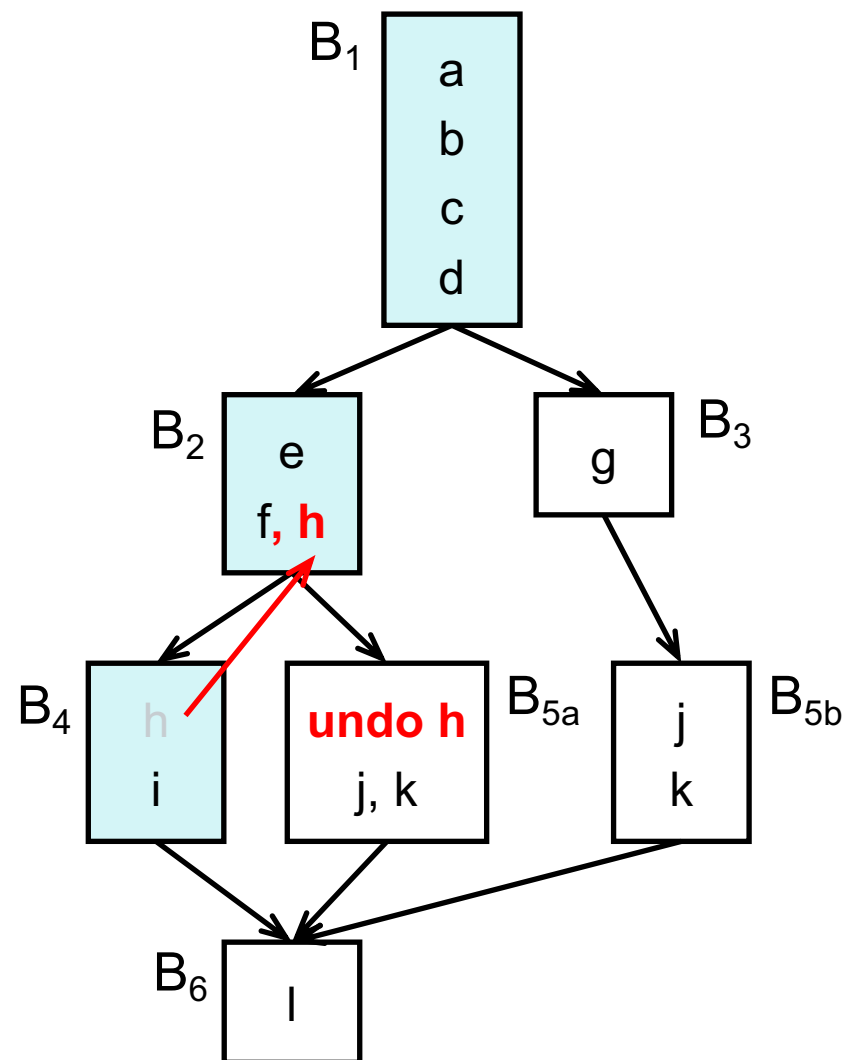
Global Scheduling

- After scheduling an EBB path, remove it from the graph
- Then schedule the next, until all EBB paths are scheduled



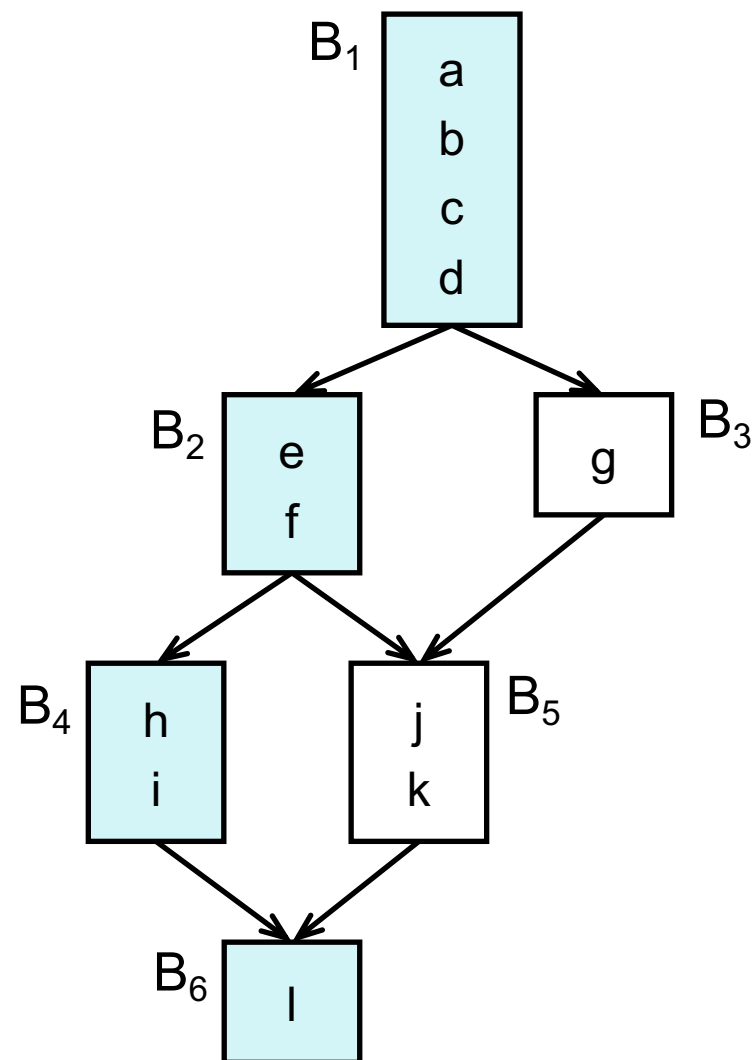
Context Problem at Join Points

- Join points
- Context Problem:
 - From B2 to B5, Undo h
 - From B3 to B5, Undo h changes the code semantics
- **Clone blocks to address the context problem**



Trace Scheduling

- Profile and schedule the hot path
- Profiling-guided optimization
- Just-in-time compiler (that in JVM)
- What is JIT?

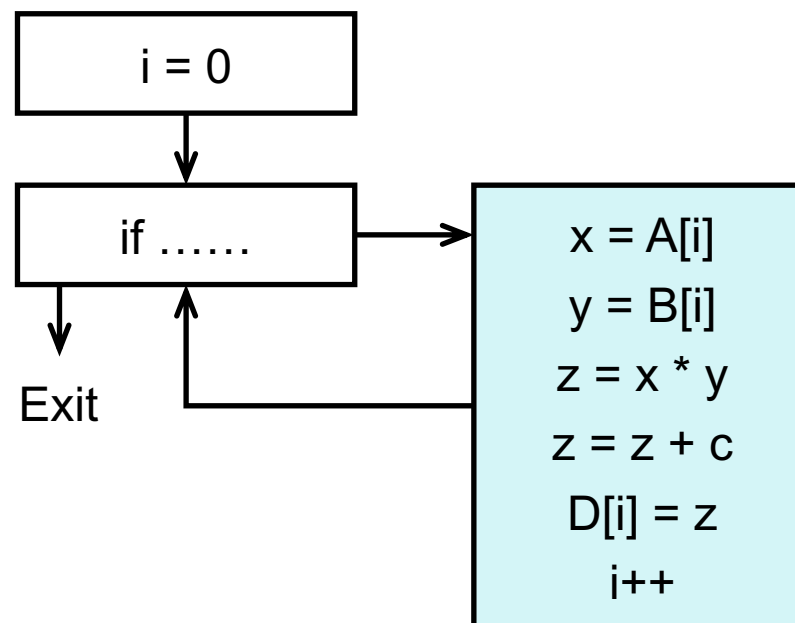


PART IV: Software Pipelining

How about Loops?

- We can schedule the loop body as a single block
- Miss cross-iteration parallelism

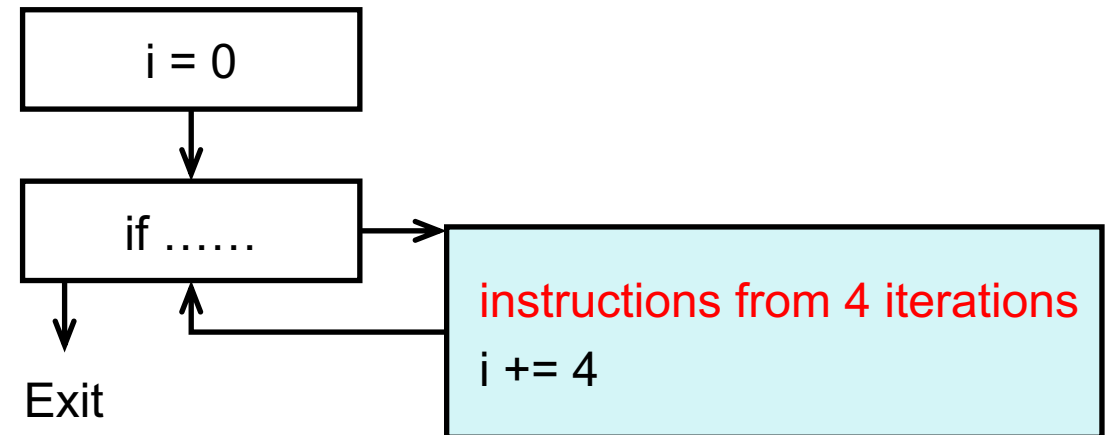
```
for (i = 0; .....; i++)  
    D[i] = A[i] * B[i] + c
```



Loop Unrolling Helps!

- We still schedule the loop body as a single block
- Let us schedule instructions across multiple loop iterations

```
for (i = 0; .....; i += 4)
    D[i]    = A[i]    * B[i]    + c
    D[i+1]  = A[i+1]  * B[i+1]  + c
    D[i+2]  = A[i+2]  * B[i+2]  + c
    D[i+3]  = A[i+3]  * B[i+3]  + c
```

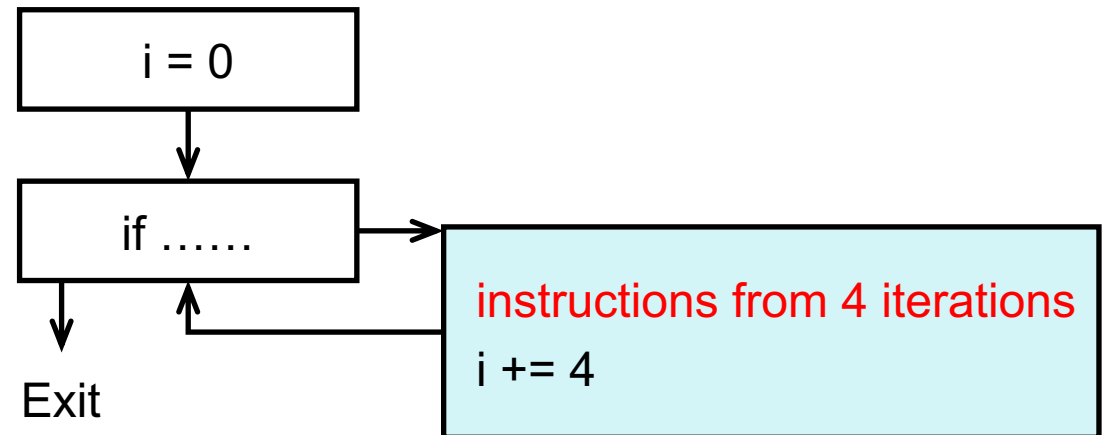


Problems of Loop Unrolling

- We cannot unroll too many times, which
- I. makes code size larger
 - cache pressure
 - register pressure
 - (slows down the code)
- II. and does not break the boundary of loop iteration

```

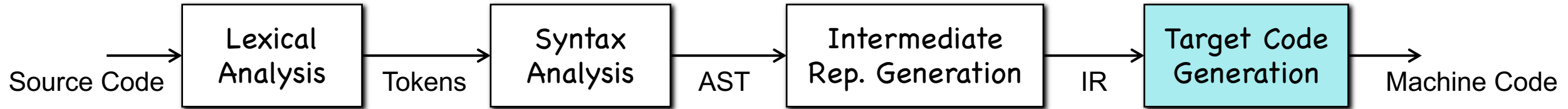
for (i = 0; .....; i += 4)
    D[i]    = A[i]    * B[i]    + c
    D[i+1]  = A[i+1]  * B[i+1]  + c
    D[i+2]  = A[i+2]  * B[i+2]  + c
    D[i+3]  = A[i+3]  * B[i+3]  + c
    
```



Software Pipelining

- Break the boundary of loop iterations
- Regard a loop as a whole, not separate blocks
- More chances of parallelism **without** bloating the code
- ***Refer to Chapter 10, the Dragon book***
- ***No Silver Bullet!***

Summary



Can we generate code to leverage modern CPUs for better instruction-level parallelization? **Yes! Possible!**

- Scheduling a basic block
- Scheduling extended basic blocks
- Scheduling loops (optional)